Distinction Graph Machines: Refine-or-Repair Runtime Learning

Anonymous

Abstract

Runtime learners often treat every surprise as an update signal. This paper argues that the first question should instead be which part of runtime excess risk is being observed. Given current information $\mathcal{F}_t$, predictor f_t, a candidate refinement $\mathcal{G} \supseteq \mathcal{F}_t$, and Bayes risk $R^\star$ under an information structure, excess risk decomposes into a repair gap $R(f_t) - R^\star(\mathcal{F}_t)$ and a refinement gap $R^\star(\mathcal{F}_t) - R^\star(\mathcal{G})$. The first is stale local state under adequate information; the second is missing information.

We formalize this decomposition with information maps, partitions, and sigma-algebras, prove the architecture-independent missing-distinction obstruction, and show that the minimal corrective refinement is the join with the missing evidence. We then introduce Distinction Graph Machines (DGM), a runtime learner whose nodes repair local memories and whose edges create the distinctions that future reads may condition on. For language models, DGM-Embedding implements the same refine-or-repair rule as a detached, forward-only next-token memory layer with no inference-time backpropagation. Controlled hidden-regime sequence experiments show the intended separation: repair-only memories collapse incompatible consequences, while refine+repair DGM creates runtime concepts and recovers the needed distinctions.

Contents

1 Introduction **1**

2 Repair and Refinement Gaps **4**
2.1 Information maps and fibers . . . 4
2.2 Measurable predictors . . . 5
2.3 Bayes risk under information . . . 5
2.4 Repair and refinement . . . 5
2.5 Contradiction inside a cell . . . 6
2.6 Minimal refinement . . . 6
2.6.1 The finite refinement rule . . . 7
2.6.2 The sigma-algebra form . . . 7

3 DGM: Nodes Repair, Edges Distinguish **7**
3.1 Why a graph, not a tree? . . . 7
3.2 State: anchors, centroids, memories, and edges . . . 8
3.3 Edge-gated prediction . . . 9
3.4 Refine or repair on a graph . . . 11
3.5 Canonical local memory . . . 14
3.6 From DGM to DGM-Embedding . . . 15

4 DGM-Embedding: A Runtime Memory Layer for Language Models **15**
4.1 Hidden-state concepts . . . 15
4.2 Logit correction . . . 16
4.3 Read before observe . . . 16
4.4 Sequential detached training . . . 17

5 Guarantees **18**

6 Experiments: When Refinement Matters **19**
6.1 Hidden-regime sequence prediction . . . 19
6.2 Repair-only ablation . . . 20
6.3 Controlled contextual XOR . . . 20
6.4 Synthetic next-token stream . . . 20

7 Related Work **21**

8 Conclusion **22**

A Appendix Roadmap **28**

B Limitations **28**

C Related Work **29**
C.1 Methods that repair without refining . . . 29
C.2 Methods that refine without local repair . . . 29
C.3 Methods that route without runtime concept creation . . . 30
C.4 DGM's position . . . 30

D DGM-Embedding Extensions **30**
D.1 Trees as a hard-routing special case . . . 30
D.2 DGM data-structure cost . . . 31
D.3 Ridge local memory . . . 31
D.4 Training modes for representation drift . . . 31
D.5 Non-canonical adapter extension . . . 32
D.6 Cost and systems notes . . . 32
D.7 Closed-form local repair . . . 33

E Additional Computational Checks **33**
E.1 Online image streams . . . 33
E.2 Frozen-embedding adapter check . . . 34
E.3 Projection-memory residual identity . . . 34

F The Value of a Distinction **34**
F.1 Risk monotonicity . . . 35
F.2 Log loss . . . 35
F.3 Squared loss . . . 35
F.4 Finite empirical scores . . . 35

G Proof Sketches **36**

H From Distinctions to Projection Memory **41**
H.1 One-hot partition memory . . . 41
H.2 CPM conflict as missing distinction . . . 41
H.3 The hierarchy . . . 42

I Minimal Refinement Proofs **42**

J Value of Distinction Proofs **44**

K DGM Algorithm and Projection-Bridge Proofs **45**

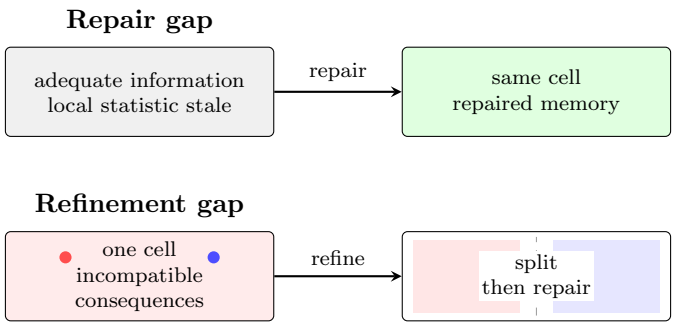

Figure 1: Runtime excess risk separates into a repair gap and a refinement gap. Stale state should be repaired under adequate information; incompatible consequences inside one information cell require refinement.

1 Introduction

At time t, a runtime learner has two objects: an information map $I_t : \Omega \to \mathcal{Z}_t$, which determines what situations the learner can distinguish, and a predictor $f_t = g_t \circ I_t$, which determines what it currently does with those distinctions. Let $\mathcal{F}_t = \sigma(I_t)$, and let $R^\star(\mathcal{F})$ be the Bayes risk achievable by predictors measurable with respect to $\mathcal{F}$. For any candidate refinement $\mathcal{G} \supseteq \mathcal{F}_t$, runtime excess risk decomposes as

$$R(f_t) - R^\star(\mathcal{G}) = \underbrace{R(f_t) - R^\star(\mathcal{F}_t)}_{\text{repair gap}} + \underbrace{R^\star(\mathcal{F}_t) - R^\star(\mathcal{G})}_{\text{refinement gap}}. \tag{1}$$

The repair gap is error that can be removed while preserving the current information. The refinement gap is error caused by information that is too coarse. DGM is built around estimating which gap is responsible before it writes new runtime state.

A deployed language model gives the intuition. Early in a document, "Jordan" names a person; later, it names a country. A cache can store both observations, a nearest-neighbor memory can retrieve similar prefixes, and a test-time optimizer can move a hidden state. None of these operations fixes the structural error if the current information map gives both cases the same runtime label, $I_t(\omega) = I_t(\omega')$, while their next-token consequences differ. The missing object is not a better local update; it is the distinction that separates the two fibers of I_t.

Many runtime-learning mechanisms implement one dominant response to surprise. Cache and kNN language models store or retrieve more examples [GCJ17, KLJ+20]. Fast weights and delta-rule linear attention repair associative state [BHM+16, SIS21]. Test-time training and adaptation optimize an inference-time surrogate [SWL+20, WSL+21, SLD+25]. Online trees refine leaves by split scores [BFOS84, DH00]. Sparse MoE routes to trained experts [SMM+17, FZS22]. Adaptive category and prototype systems, including ART 1 and ART 2, create or move categories [CG87b, CG87a, Koh90, Fri95, SSZ17]. DGM is not a claim that these ingredients are new. The claim is that a runtime learner should first decide which operation is legal: repair within the current information, or refine the information itself.

The separation is structural. A learner's information state can be represented by an information map, a partition, or a sigma-algebra, and its admissible predictors are precisely the measurable functions with respect to that information. If I_t identifies situations that the consequence separates, no predictor over the current information can satisfy both. More computation inside the same fiber cannot remove the contradiction. This is the paper's central diagnosis: a positive refinement gap is not a failed optimization step; it is a certificate that the information is too coarse.

Method family	What it does when surprised	Why that is incomplete
Cache/kNN-LM	store or retrieve more examples	does not decide whether a new concept is needed
Fast weights / local memories	repair associative state	cannot fix missing distinctions
TTT/TTA	optimize a test-time objective	treats adaptation as surrogate-loss descent
Online trees	split leaves	rooted hierarchy; no local repair memory at graph concepts
MoE / PKM	route to trained capacity	experts or keys are static, not runtime-created concepts
ART systems / prototypes	create or move categories	lacks edge-generated local repair structure

Representative references. Cache and kNN-LM: [GCJ17, KLJ+20]; fast weights and local memories: [BHM+16, SIS21]; TTT/TTA: [SWL+20, WSL+21, SLD+25]; online trees: [BFOS84, DH00]; MoE and PKM: [SMM+17, FZS22, LSR+19]; ART, ART 2, and prototypes: [CG87b, CG87a, Koh90, Fri95, SSZ17].

Table 1: Representative runtime-learning mechanisms and the single dominant update they apply when surprised. DGM's claim is not that these mechanisms are absent from prior work, but that runtime learning needs an explicit decision between refining the information structure and repairing state inside it.

The formal core is then minimal information refinement. In a finite space, adding a binary distinction cuts exactly the old atoms crossed by that distinction and leaves all other atoms unchanged. In a general measurable space, the same operation is the join

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(Z_t).$$

This is the smallest information update that makes the new evidence observable, using standard language from measure-theoretic probability and statistical decision theory [Bil95, Kal02, Bla53, Ber85]. Population information cannot hurt, but finite-sample estimation can. Thus a useful runtime learner should refine only when a statistically guarded gain test justifies the new distinction; value identities for log and squared losses are deferred to Appendix F.

If the target separates states that the information map identifies, repair is impossible.

Distinction Graph Machines implement this theorem-to-algorithm path as a data structure. If contradictions require distinctions, the learner needs runtime concepts. If concepts must remain stable, each concept needs a frozen anchor. If retrieval must adapt, each concept also needs a movable centroid. If distinctions are local, the structure should add pairwise graph edges rather than rewrite a global tree. If concepts have state, each node should carry a repairable memory. A DGM therefore maintains anchored concepts, movable retrieval centroids, local memories, and distinction edges. Edges are not metadata: they are binary generators that define observable distinctions and gate future routing. When a consequence arrives, DGM either repairs local memory under fixed anchors and edges, or creates a new anchored concept and edges after a guarded refinement test.

Terminology. Throughout the paper, *DGM* denotes the runtime machine as a whole. A *concept* is a graph node. Each concept has a frozen *anchor* a_j, a movable *centroid* c_j, and a repairable *memory* M_j. A *distinction edge* is a binary generator stored between concepts. We use *hard*

route for the exact anchor-defined selector used in the theory, and *soft read* for the differentiable edge-gated implementation. We reserve *prototype* mostly for related work and for standard nearest-prototype terminology.

At one timestep, the whole mechanism has the structure in Algorithm 1.

Algorithm 1 RefineOrRepairStep (structural template)

```
Input: runtime state and current context
Read: use the current concepts, edges, and memories to predict
Observe: receive the consequence for this context
if the current concept is adequate then
    Repair: update local state while keeping anchors and edges fixed
else
    Refine: add a concept and distinction edges, then initialize local state
end if
return updated runtime state
```

The template deliberately omits retrieval rules, edge gates, scoring buffers, and memory forms. The algorithms in Section 3 fill in those choices. The theory supplies the two quantities behind the template: the repair gap says how much can be gained under fixed information, and the refinement gap says how much is lost because the current information is too coarse. Thus DGM is not a generic memory with a growth heuristic; it is a runtime implementation of a decision-theoretic question: is this error a repair gap or a refinement gap?

For language models, the canonical layer is DGM-Embedding. Hidden-state concepts store next-token embedding memories and produce sparse logit corrections. The read path is differentiable, so slow model parameters can learn to use DGM memory. The write path is detached and forward-only: after observing the next token, the layer repairs selected local memories or refines the concept graph without inference-time backpropagation. Adapter variants, projection-style repair, and CPM are useful extensions, but they are not the main object and are deferred to the appendix.

Contributions. The contribution is a method paper built around an error decomposition.

- **Error decomposition.** We decompose runtime excess risk into a repair gap at fixed information and a refinement gap from insufficient information.
- **Missing-distinction obstruction.** We show that if consequences separate states identified by I_t, no repair update preserving $\sigma(I_t)$ can solve the error.
- **Refine-or-repair algorithm.** We introduce DGM, a graph-based runtime learner whose nodes repair local memories and whose edges create consequence-forced distinctions.
- **Language-model layer.** We instantiate DGM as DGM-Embedding, with differentiable sparse reads and detached forward-only writes for next-token memory.
- **Controlled evidence.** We evaluate settings where the same surface state has regime-dependent consequences, showing the separation between repair-only and refine+repair behavior.

Scope of the formal claims. The theorem-level question is not whether DGM is the globally optimal runtime architecture. It is: given a current information structure $\mathcal{F}$, what part of the error is repairable at fixed information, what part requires refinement, and what is the least information that must be added once a new distinction is required? Section 2 answers this question as a short diagnostic theory, and Section 3 gives the finite-sample guarded algorithm. DGM and DGM-Embedding are algorithmic realizations of this refine-or-repair principle; optimization guarantees for deep nonconvex backbones and global search over all possible distinctions are outside the theorem statements.

Organization. Section 2 gives the repair/refinement gap decomposition, missing-distinction obstruction, and minimal refinement rule. Section 3 turns the diagnosis into Distinction Graph Machines. Section 4 gives DGM-Embedding for language models. Section 5 states the core guarantees, Section 6 evaluates when refinement matters, and Section 7 positions DGM relative to prior runtime learners.

2 Repair and Refinement Gaps

The theory in the main text has one job: to diagnose whether an observed runtime error is a repair gap or a refinement gap. We use information maps, partitions, and sigma-algebras to make indistinguishability precise. A predictor can use only distinctions present in the current information structure.

2.1 Information maps and fibers

At time t, let

$$I_t : \Omega \to \mathcal{Z}_t$$

be the learner's current information map. The value $I_t(\omega)$ is the runtime state label available to the learner. Two worlds ω, ω' are indistinguishable under the current representation when $I_t(\omega) = I_t(\omega')$. The fiber, or cell,

$$I_t^{-1}(z) = \{\omega : I_t(\omega) = z\}$$

is the set of worlds currently treated as the same situation. The sigma-algebra generated by the current information is

$$\mathcal{F}_t = \sigma(I_t).$$

In the finite setting, this is equivalent to a partition

$$\mathcal{P} = \{B_1, \ldots, B_n\}$$

of Ω into nonempty disjoint cells. A partition $\mathcal{Q}$ refines $\mathcal{P}$ if every cell of $\mathcal{Q}$ is contained in a cell of $\mathcal{P}$. An event $E \subseteq \Omega$ is $\mathcal{P}$-measurable if it is a union of cells of $\mathcal{P}$. Thus a partition is not only notation. It is the learner's current quotient of the world into distinguishable cases.

For sigma-algebras $\mathcal{F}, \mathcal{G} \subseteq \mathcal{A}$, the inclusion $\mathcal{F} \subseteq \mathcal{G}$ means that $\mathcal{G}$ is at least as informative as $\mathcal{F}$. The join $\mathcal{F} \vee \mathcal{G}$ is the smallest sigma-algebra containing both. For a random variable Z, $\sigma(Z)$ is the sigma-algebra generated by Z; for an event E, $\sigma(E) = \{\varnothing, E, E^c, \Omega\}$.

2.2 Measurable predictors

A measurable predictor is what the learner is allowed to compute from its current distinctions. With information map I_t, the runtime predictor has the form

$$f_t(\omega) = g_t(I_t(\omega))$$

for some downstream rule g_t. Equivalently, f_t is $\mathcal{F}_t$-measurable. In a finite partition, a prediction or action $f : \Omega \to A$ is $\mathcal{P}$-measurable if it is constant on each cell:

$$\omega, \omega' \in B \in \mathcal{P} \quad \Longrightarrow \quad f(\omega) = f(\omega').$$

Equivalently, f can depend only on which cell contains the current world state. In the sigma-algebra setting, admissible predictors are $\mathcal{F}$-measurable functions. This is the formal reason a missing distinction cannot be repaired by a more clever local update inside the same cell: the update still returns a function on the same quotient.

2.3 Bayes risk under information

Bayes risk under information is the best possible performance given the distinctions currently available. Let $Y : \Omega \to \mathcal{Y}$ be a target random variable, let A be an action space, and let $L : \mathcal{Y} \times A \to \mathbb{R}$ be a loss for which the expectations below are well-defined. The Bayes risk available under information $\mathcal{F}$ is

$$R_L(\mathcal{F}) = \inf\{\mathbb{E}[L(Y, a)] : a : \Omega \to A,\ a \text{ is } \mathcal{F}\text{-measurable}\}. \tag{2}$$

Thus $R_L(\mathcal{F})$ is the best population risk achievable by a predictor that can use only distinctions in $\mathcal{F}$, following the standard decision-theoretic view of actions, losses, Bayes risk, and comparison of information structures [Bla53, Ber85].

2.4 Repair and refinement

For a concrete predictor f, write its risk as

$$R_L(f) = \mathbb{E}[L(Y, f)].$$

A *repair* update preserves information:

$$\mathcal{F}_{t+1} = \mathcal{F}_t.$$

At the population level, the best possible repair can only move within the class of $\mathcal{F}_t$-measurable predictors, so its limit is $R_L(\mathcal{F}_t)$. The corresponding repair gap is

$$\mathrm{RepGap}_L(f_t, \mathcal{F}_t) = R_L(f_t) - R_L(\mathcal{F}_t). \tag{3}$$

A *refinement* update changes information:

$$\mathcal{G} \supsetneq \mathcal{F}_t.$$

For a candidate refinement $\mathcal{G}$, the refinement gap is

$$\mathrm{RefGap}_L(\mathcal{F}_t, \mathcal{G}) = R_L(\mathcal{F}_t) - R_L(\mathcal{G}). \tag{4}$$

The repair gap is residual error under the current information. The refinement gap is the value of distinctions not yet present in the current information.

Theorem 2.1 (Runtime error decomposition)**.** *Let f_t be $\mathcal{F}_t$-measurable and let $\mathcal{G} \supseteq \mathcal{F}_t$ be a candidate refinement. Then*

$$R_L(f_t) - R_L(\mathcal{G}) = \mathrm{RepGap}_L(f_t, \mathcal{F}_t) + \mathrm{RefGap}_L(\mathcal{F}_t, \mathcal{G}). \tag{5}$$

Both terms are nonnegative.

Proof. Add and subtract $R_L(\mathcal{F}_t)$. The repair gap is nonnegative because $R_L(\mathcal{F}_t)$ is the infimum over all $\mathcal{F}_t$-measurable predictors, including f_t. The refinement gap is nonnegative because every $\mathcal{F}_t$-measurable predictor is also $\mathcal{G}$-measurable when $\mathcal{F}_t \subseteq \mathcal{G}$. □

2.5 Contradiction inside a cell

The hard-label obstruction is the clearest special case of a positive refinement gap. If a cell contains states that require different outputs, the current information structure does not merely have a bad parameter value; it lacks the expressive degree of freedom needed to represent the target.

Corollary 2.2 (Contradiction requires distinction)**.** *Let $I : \Omega \to \mathcal{Z}$ be an information map and let $y : \Omega \to A$ be a required output. Suppose $I(\omega) = I(\omega')$ but $y(\omega) \neq y(\omega')$. Then no predictor of the form $f = g \circ I$ satisfies both $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$.*

This corollary is deliberately architecture-independent. It applies to a neural cache, a local adapter, a fast-weight memory, a nearest-neighbor system, or a test-time optimizer whenever the update is constrained to the current information. If the relevant states are not separated by I, no repair rule inside the cell can represent the required behavior. The contradiction is evidence that the cell should be split.

Example 2.3 (One prefix, two consequences)**.** *Suppose the current runtime concept groups two prefixes because their surface form is the same. In one hidden regime the next token should be "red"; in another it should be "blue". Any memory attached to the single concept must store one consequence distribution for both regimes. A repair update can move that distribution, but it cannot make it simultaneously put all mass on "red" and all mass on "blue". The missing operation is to split the concept by a distinction correlated with the hidden regime, then repair the two resulting memories separately.*

Notation. All probability spaces are denoted by $(\Omega, \mathcal{A}, \mathbb{P})$ unless the finite case is stated explicitly. Random variables are functions on Ω. We use standard measure-theoretic probability notation for generated sigma-algebras, joins, measurability, and conditional expectation [Bil95, Kal02]. For vectors $x \in \mathbb{R}^d$, lengths use the ℓ_2 norm $\|x\|_2$. For matrices, $\|M\|_F$ is the Frobenius norm. Inner products are written $\langle x, y \rangle$.

2.6 Minimal refinement

Once a contradiction shows that the current atom is too coarse, the next question is not how to grow the largest possible model. It is: what is the least information that must be added? The primitive operation in this paper is minimal refinement. The learner has an information structure and receives evidence that a new event or variable must become observable. The correct update adds that observability and nothing else. The sigma-algebra language is standard measure-theoretic probability, while the finite partition language is the corresponding finite special case [Bil95, Kal02]. We state the finite theorem first because it is the clearest form of the idea: split exactly the atoms crossed by the evidence.

2.6.1 The finite refinement rule

Let $\mathcal{P}$ be a finite partition and let $E \subseteq \Omega$ be a new binary distinction. Define

$$\begin{aligned}\mathcal{P}_E = &\{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \\ &\cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}.\end{aligned} \tag{6}$$

This notation simply keeps the nonempty pieces obtained by cutting each old atom by E.

Theorem 2.4 (Minimal finite refinement)**.** *Let $\mathcal{P}$ be a finite partition of Ω and let $E \subseteq \Omega$. The partition $\mathcal{P}_E$ is the unique minimal partition that refines $\mathcal{P}$ and makes E measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

The theorem gives a canonical update. We do not search over all refined representations; the evidence itself determines the least one. An atom untouched by E is left unchanged. An atom crossed by E is split into the two pieces that the evidence can now distinguish. This is the formal version of "create the missing distinction and nothing else."

2.6.2 The sigma-algebra form

For general spaces, the same operation is the join of sigma-algebras. If the current information is $\mathcal{F}_t$ and the evidence is a variable Z_t, the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(Z_t). \tag{7}$$

Theorem 2.5 (Minimal sigma-algebra refinement)**.** *Let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra and let Z be a random variable. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner may condition on. The contradiction theorem in Section 2 says when such a change is necessary; minimal refinement says how little information needs to be added.

3 DGM: Nodes Repair, Edges Distinguish

The theory so far says what the learner must decide: is the observed error a repair gap at fixed information, or a refinement gap caused by missing distinctions? A runtime model then needs a data structure that can create distinctions without destroying the local state that remains useful. A tree is a useful finite special case, but it is not the most natural runtime structure for neural hidden states. A tree makes every decision through one rooted path. Hidden-state conflicts are often local: a new concept may need to be separated from a few confused neighbors without rewriting a global hierarchy.

3.1 Why a graph, not a tree?

DGM uses a graph of concepts. Local memories live at nodes, and distinctions live on edges between concepts that have been confused. The central object is

$$\text{DGM} = \text{graph-generated information structure} + \text{local memory repair}.$$

This section defines that object. The graph is not metadata: its edges generate the finite information structure and gate the read path. The partition and generated-sigma-algebra language follows standard finite measurable-space constructions [Bil95, Kal02]. The main-text statements below give the operational theorem forms; full proofs are collected in Appendix K.

The minimal DGM loop has four moving parts. First, anchors and edges define what the machine can distinguish. Second, centroids retrieve a small candidate set. Third, local memories predict through edge-gated routing. Fourth, after observing the consequence, the machine commits either a guarded repair that leaves anchors and edges fixed, or a guarded refinement that adds a new anchor and distinction edges. The rest of the section only makes these four operations precise.

3.2 State: anchors, centroids, memories, and edges

Let $\phi_\theta : \mathcal{X} \to \mathbb{R}^d$ be an encoder and write $h = \phi_\theta(x)$. A Distinction Graph Machine has runtime state

$$\mathcal{S}_t = \Big(\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t\Big), \qquad G_t = (V_t, E_t).$$

Each component has a role forced by the refine-or-repair split. A runtime concept needs a stable identity, so $a_j \in \mathbb{R}^d$ is a frozen distinction anchor. Retrieval must adapt as more examples arrive, so $c_j \in \mathbb{R}^d$ is a movable retrieval centroid. State inside an adequate concept must be repairable, so M_j is a local memory and n_j is a usage count. If a concept is created because it was confused with existing concepts, that confusion should not disappear after the update; it becomes an edge in E_t. A new concept is initialized with $a_j = c_j = h$. Subsequent repair may move c_j, but anchors are fixed once created, so old distinctions keep their original semantics.

An edge $e = (i, j)$ stores a linear distinction

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j),$$

with side labels $s_{e,i} = +1$ and $s_{e,j} = -1$. Its binary generator is

$$g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}.$$

The edge set induces an information structure. Define the graph signature

$$\sigma_t^G(h) = \big(g_e(h)\big)_{e \in E_t} \in \{0,1\}^{E_t}.$$

Two hidden states are indistinguishable by the current graph if their signatures agree. The nonempty level sets

$$A_a^t = \{h : \sigma_t^G(h) = a\}, \qquad a \in \{0,1\}^{E_t},$$

form a finite partition $\mathcal{P}_t^G$ of hidden-state space, and the generated sigma-algebra is $\sigma(\mathcal{P}_t^G)$. Adding a new edge is therefore a join with one binary generator:

$$\mathcal{P}_{t+1}^G = \mathcal{P}_t^G \vee \sigma(g_e).$$

Anchors generate the distinction skeleton. Centroids and local memories are adaptive statistics attached to cells or sparse neighborhoods of this graph-generated partition.

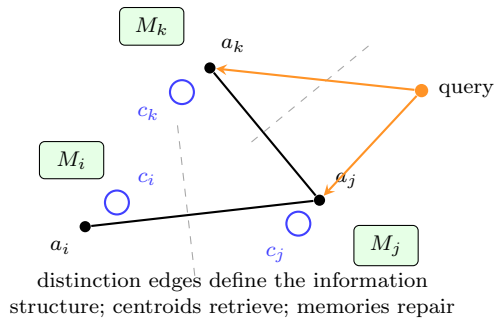

Figure 2: DGM state. Anchors are stable concept identities, centroids are movable retrieval statistics, distinction edges generate the information structure, and concept memories are repaired after a concept is judged adequate.

Theorem 3.1 (Anchor distinction separates its endpoints)**.** *If $a_i \neq a_j$ and u_e, b_e are defined as above for edge $e = (i, j)$, then*

$$u_e^\top a_i - b_e = \frac{1}{2}\|a_i - a_j\|_2, \qquad u_e^\top a_j - b_e = -\frac{1}{2}\|a_i - a_j\|_2.$$

Thus each edge distinction separates the two anchors that caused it.

Theorem 3.2 (DGM graph generates a finite information structure)**.** *For any finite edge set E_t, equality of signatures $\sigma_t^G(h)$ is an equivalence relation whose classes form a finite partition $\mathcal{P}_t^G$. Adding one edge g refines the partition by $\mathcal{P}_t^G \vee \sigma(g)$.*

3.3 Edge-gated prediction

Given h, the soft implementation first computes retrieval distances $d_j(h) = \|h - c_j\|_2^2$ to movable centroids. A centroid-only top-k set is not enough for the theory: if the hard eligible anchor never enters the candidate set, edge gates cannot recover it. We therefore use a recall-oriented candidate set

$$N(h) = N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}\Big(N_{k_c}^{\text{centroid}}(h)\Big),$$

with duplicates removed and with an optional budget cap after the union. The first term gives adaptive locality, the second protects the anchor-defined hard route, and the third recovers concepts connected by recent consequence conflicts. In small exact implementations one may simply take $N(h) = \{1, \ldots, N_t\}$. If $I_t(j)$ is the set of edges incident to concept j, define the soft compatibility of h with concept j by

$$\log \chi_j(h) = \frac{1}{\sqrt{\max\{1, |I_t(j)|\}}} \sum_{e \in I_t(j)} \beta_e \log \text{sigmoid}\left(\frac{s_{e,j}(u_e^\top h - b_e)}{\tau_e}\right),$$

where structural distinction edges have $\beta_e \in (0, 1]$ and the empty sum gives $\chi_j(h) = 1$. The degree normalization avoids systematically suppressing high-degree concepts while preserving hard rejection in the low-temperature limit. The edge-gated routing weights are

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T)\chi_j(h)}{\sum_{r \in N(h)} \exp(-d_r(h)/T)\chi_r(h)}, \qquad j \in N(h). \tag{8}$$

The prediction is the local mixture

$$\hat{y} = \sum_{j \in N(h)} \alpha_j(h) M_j(h).$$

The hard route makes the information-structure connection explicit. A concept j is eligible for h if every incident edge bit places h on j's recorded side:

$$\begin{cases} g_e(h) = 1, & s_{e,j} = +1, \\ g_e(h) = 0, & s_{e,j} = -1, \end{cases} \qquad \text{for every } e \in I_t(j).$$

This convention assigns boundary points to the nonpositive side because $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$. The exact hard route used in the refinement theory routes to the nearest eligible anchor, falling back to the nearest anchor if no eligible concept is available. The soft read in (8) is the trainable implementation: it may retrieve by movable centroids, but those centroids are retrieval statistics rather than formal information cells.

The distinction partition alone is not the whole exact hard route. Anchor distances add a stable order geometry. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be the finite sigma-algebra generated by all pairwise anchor-distance comparison events

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}, \qquad 1 \le i, j \le N_t,$$

with deterministic tie-breaking. The hard routing skeleton is measurable with respect to

$$\mathcal{F}_t^{\text{DGM}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}.$$

Equivalently, the exact DGM information map can be written as

$$I_{\mathcal{S}_t}(h) = \left((g_e(h))_{e \in E_t}, \text{HardRoute}_{\mathcal{S}_t}(h)\right), \qquad \mathcal{F}_t^{\text{DGM}} = \sigma(I_{\mathcal{S}_t}).$$

Thus the exact information-structure theory applies to the anchor-based hard route. The soft read keeps the same graph skeleton but may use movable centroids and smooth gates for locality, so it should be read as a differentiable engineering relaxation of the anchor-generated information map rather than as a new formal refinement.

Theorem 3.3 (Anchor-based hard routing is graph-plus-anchor-order measurable)**.** *With finite E_t, finite frozen anchors $a_1, \ldots, a_{N_t}$, and deterministic tie-breaking, the exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{DGM}} = \mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.*

The next statement is analytic rather than structural: it applies to whichever fixed distances a soft read uses. With anchor distances it recovers the exact hard route; with centroid distances it describes the behavior of the soft retrieval approximation. It also exposes the candidate-recall requirement that links the formal hard route to the sparse implementation.

Condition 3.4 (Candidate recall)**.** *Let $j^\star(h)$ be the unique concept selected by the exact hard route for query h. The sparse soft read satisfies candidate recall at h if*

$$j^\star(h) \in N(h).$$

If this condition fails, the soft read has no guarantee of selecting the hard route, regardless of how sharp the edge gates are.

Theorem 3.5 (Soft routing approaches hard routing)**.** *Assume candidate recall holds for h. If the hard-route selected concept is unique within $N(h)$ with distance margin $\delta > 0$, then its soft routing weight tends to one exponentially as $T \to 0$. If every wrong-side candidate violates some incident structural edge with weight at least $\beta_{\min} > 0$, signed margin at least $\delta_e > 0$, and degree at most $D_{\max}$, then each wrong-side compatibility decays uniformly after degree normalization. If at least one eligible reference candidate has compatibility bounded below and the wrong-side/reference distance-score ratios are bounded, then total wrong-side routing mass vanishes as the edge temperature $\tau_e \to 0$. Without candidate recall, the theorem gives no correctness guarantee for an omitted hard-route concept.*

Algorithm 2 PREDICT for a Distinction Graph Machine

Input: state $\mathcal{S}_t = (\{a_j, c_j, M_j, n_j\}_{j=1}^{N_t}, G_t)$, encoder ϕ_θ, example x
$h \leftarrow \phi_\theta(x)$
$N(h) \leftarrow N_{k_c}^{\text{centroid}}(h) \cup N_{k_a}^{\text{anchor}}(h) \cup \text{GraphNeighbors}(N_{k_c}^{\text{centroid}}(h))$
for $j \in N(h)$ **do**
    compute edge compatibility $\chi_j(h)$ from incident distinctions
    compute edge-gated weight $\alpha_j(h)$ by (8)
end for
return $\sum_{j \in N(h)} \alpha_j(h) M_j(h)$

3.4 Refine or repair on a graph

After observing consequence y, DGM estimates empirical versions of the two gaps in Theorem 2.1. A repair candidate keeps the DGM information map fixed and updates only local state. A refinement candidate changes the information map by adding an anchored concept and distinction edges. The accepted operation is the one whose held-out gain justifies its cost and safety margin.

Let $j^\star$ be the highest-weight concept under (8). Define local surprise

$$\rho_t = L(y, M_{j^\star}(h)).$$

Define neighborhood consequence disagreement on $N(h)$ by the task-appropriate local prediction divergence. For classification one may use

$$D_t = \max_{i,r \in N(h)} \text{TV}(p_i(\cdot \mid h), p_r(\cdot \mid h)),$$

where p_j is the distribution represented by memory M_j. For regression one may use

$$D_t = \max_{i,r \in N(h)} \|\mu_i(h) - \mu_r(h)\|_2 ,$$

and for language modeling the log-loss surprise $-\log p_{j^\star}(x_{t+1} \mid x_{\leq t})$ is the local disagreement signal. These quantities are proposal signals: they decide which candidates are worth scoring, not the formal meaning of repair or refinement.

All gap estimates below are computed on a held-out scoring buffer $\mathcal{B}_t^{\text{score}}$. Refinement proposals may use a disjoint proposal buffer $\mathcal{B}_t^{\text{prop}}$ together with the current observation; the scoring buffer is held out from proposal generation. A practical implementation can maintain the split by reservoir sampling incoming observations into the two buffers. If an implementation reuses one buffer for both proposal and scoring, the empirical rule still defines a heuristic, but Theorem 3.8 applies to the sample-split version.

The edges created by a refinement are attached to a concrete confused-neighbor set. Let $\mathrm{Div}_{\mathcal{Y}}(M_i, M_r; h)$ denote a task-appropriate consequence divergence, such as total variation for classifiers, Euclidean mean distance for regression memories, or next-token distribution divergence for language models. Define

$$\mathrm{Confused}(h, y) = \{i \in N(h) : L(y, M_i(h)) > \tau_{\mathrm{conflict}} \text{ or } \mathrm{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h) > \tau_{\mathrm{disagree}}\}.$$

When a budget of at most q new edges is used, DGM takes $\mathrm{Confused}_q(h, y)$ to be the q highest-scoring members of this set under

$$\mathrm{score}_i(h, y) = \alpha_i(h) L(y, M_i(h)) + \gamma_{\mathrm{div}} \mathrm{Div}_{\mathcal{Y}}(M_i, M_{j^\star}; h).$$

If the set is empty but refinement is otherwise accepted, the implementation falls back to the highest-score candidates in $N(h) \setminus \{j^\star\}$. Thus "confused neighbors" is an operator, not an informal phrase.

Repair is guarded rather than blindly committed. The implementation proposes a small set of repair candidates, such as a memory-only update, a memory-plus-centroid update, and one or more reduced centroid steps:

$$\mathcal{C}_{\mathrm{repair}}(\mathcal{S}_t, h, y) = \{\mathcal{S}_t^{\mathrm{mem}}, \mathcal{S}_t^{\mathrm{mem+centroid}}, \mathcal{S}_t^{\mathrm{shrink}}, \mathcal{S}_t\}.$$

Each candidate keeps anchors and edges fixed, so it preserves $I_{\mathcal{S}_t}$ and $\mathcal{F}_t^{\mathrm{DGM}}$. On the scoring buffer, the empirical repair gain is

$$\widehat{\Delta}_{\mathrm{repair}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\mathrm{score}}) - \min_{\mathcal{S}' \in \mathcal{C}_{\mathrm{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}). \tag{9}$$

DGM's guarded repair candidate is

$$\mathcal{S}_{t+1} \in \operatorname*{argmin}_{\mathcal{S}' \in \mathcal{C}_{\mathrm{repair}}} \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}) \quad \text{subject to} \quad \widehat{R}(\mathcal{S}'; \mathcal{B}_t^{\mathrm{score}}) \le \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\mathrm{score}}). \tag{10}$$

The identity state is in $\mathcal{C}_{\mathrm{repair}}$, so $\widehat{\Delta}_{\mathrm{repair}} \ge 0$. Thus monotone repair is an enforced algorithmic property, not merely a theorem assumption. The anchor $a_{j^\star}$ is never moved by repair. Consequently, local memory repair and centroid adaptation do not rewrite previously accepted edge generators.

A candidate creates

$$a_{\mathrm{new}} = h, \qquad c_{\mathrm{new}} = h, \qquad M_{\mathrm{new}} = \mathrm{InitMemory}(h, y),$$

and adds distinction edges from the new anchored concept to $\mathrm{Confused}_q(h, y)$. Let $\widehat{R}(\mathcal{S}; \mathcal{B}_t^{\mathrm{score}})$ be the empirical local loss on the scoring buffer after routing with state $\mathcal{S}$. The penalized refinement gain is

$$\Delta_{\mathrm{refine}} = \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\mathrm{score}}) - \widehat{R}(\mathcal{S}_t^{\mathrm{new}}; \mathcal{B}_t^{\mathrm{score}}) - \lambda_{\mathrm{concept}} - \lambda_{\mathrm{edge}} q. \tag{11}$$

The candidate is accepted only if it beats the repair gain by a safety margin:

$$\Delta_{\mathrm{refine}} > \max\{\widehat{\Delta}_{\mathrm{repair}}, 0\} + \tau + 2\epsilon_m \qquad \text{and} \qquad D_t > \tau_{\mathrm{disagree}}. \tag{12}$$

Here ϵ_m is a statistical safety margin for a scoring buffer of size $m = |\mathcal{B}_t^{\mathrm{score}}|$; it can be set to zero for a purely empirical implementation, or to the finite-class bound in Theorem 3.8. If the refinement test fails, DGM commits the guarded repair candidate in (10). This makes refinement a statistically guarded gap comparison rather than an unscored growth heuristic.

Algorithm 3 OBSERVE for a Distinction Graph Machine

Input: state $\mathcal{S}_t$, labeled consequence (x, y)
$h \leftarrow \phi_\theta(x)$
if $N_t = 0$ **then**
  create anchor and centroid $a_1 = c_1 = h$ with memory $\text{InitMemory}(h, y)$
  return updated state
end if
retrieve $N(h)$ and compute edge-gated routing weights
$j^\star \leftarrow \text{argmax}_{j \in N(h)} \alpha_j(h)$
$\rho \leftarrow L(y, M_{j^\star}(h))$
compute neighborhood disagreement D_t
propose repair candidates with anchors and edges fixed
compute empirical repair gain $\widehat{\Delta}_{\text{repair}}$ by (9)
if $\rho > \tau_{\text{repair}}$ or $D_t > \tau_{\text{disagree}}$ **then**
  form a candidate new anchor/centroid from h and $\mathcal{B}_t^{\text{prop}}$, with candidate edges to $\text{Confused}_q(h, y)$
  estimate penalized refinement gain Δ_{refine} on $\mathcal{B}_t^{\text{score}}$ by (11)
  if $\Delta_{\text{refine}} > \max\{\widehat{\Delta}_{\text{repair}}, 0\} + \tau + 2\epsilon_m$ and $D_t > \tau_{\text{disagree}}$ **then**
    commit the new anchored concept and its distinction edges
    return updated state
  end if
end if
if $\widehat{\Delta}_{\text{repair}} \geq 0$ **then**
  commit the best candidate satisfying the guarded repair test (10)
else
  leave state unchanged
end if
return updated state

Definition 3.6 (Guarded monotone local repair)**.** *A guarded repair operator is monotone on scoring buffer $\mathcal{B}_t^{\text{score}}$ if it commits a candidate state $\mathcal{S}_{t+1}$ only when local memories and retrieval centroids may change, anchors and edges remain fixed, and*

$$\widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) \leq \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}).$$

The identity candidate $\mathcal{S}_{t+1} = \mathcal{S}_t$ is always allowed, so the guard is feasible even when every nontrivial repair would hurt the scoring buffer.

Theorem 3.7 (Refine-or-repair objective descent)**.** *Let the local scoring objective on a fixed scoring buffer $\mathcal{B}_t^{\text{score}}$ be*

$$J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S}; \mathcal{B}_t^{\text{score}}) + \lambda_{\text{concept}} N_{\text{concept}}(\mathcal{S}) + \lambda_{\text{edge}} |E(\mathcal{S})|.$$

If repair is committed by the guarded rule (10), then $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. If a refinement adding one anchored concept and q edges is accepted only under (12), then the empirical penalized objective $J_{\mathcal{B}_t^{\text{score}}}$ decreases by more than $\widehat{\Delta}_{\text{repair}} + \tau + 2\epsilon_m$ and in particular by more than $\tau + 2\epsilon_m$.

This is a local-buffer statement. It does not imply monotone descent of a global objective when the scoring buffer changes over time; a global finiteness claim requires a common potential or validation objective.

Theorem 3.8 (Statistically safe finite-class refinement)**.** *Assume losses lie in* $[0,1]$*, the scoring buffer has* m *independent samples, and the finite candidate refinement class* $\mathcal{H}$ *is fixed before scoring, or is generated from* $\mathcal{B}_t^{\text{prop}}$ *and the current observation, independent of* $\mathcal{B}_t^{\text{score}}$*. With probability at least* $1-\delta$*, empirical and population risks are uniformly within*

$$\epsilon_m = \sqrt{\frac{\log(2(|\mathcal{H}|+1)/\delta)}{2m}}.$$

Consequently every empirical gain $\widehat{\Delta}(g)$ *differs from its population gain* $\Delta(g)$ *by at most* $2\epsilon_m$*. Hence an accepted refinement satisfying*

$$\widehat{\Delta}(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_m$$

has positive population gain after paying complexity cost and margin τ*.*

Corollary 3.9 (Bounded language-model refinement score)**.** *For language modeling, raw next-token negative log likelihood is unbounded. Fix* $L_{\max} > 0$ *and define the clipped normalized score*

$$\ell_{\text{clip}}(x_{\le t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\le t}), L_{\max}\}.$$

If refinement proposal and scoring use ℓ_{clip}*, then Theorem 3.8 applies to clipped-loss refinement gain. An accepted refinement has positive clipped population gain after cost and margin with the stated high probability. This corollary does not claim positive gain for the raw unbounded negative log likelihood without additional tail assumptions.*

Theorem 3.10 (Anchor-stable DGM information structure)**.** *Let*

$$\mathcal{F}_t^{\text{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h-a_i\|_2^2 \le \|h-a_j\|_2^2\} : 1 \le i,j \le N_t\Big).$$

If a repair step changes only retrieval centroids c_j*, local memories* M_j*, and counts* n_j*, while keeping anchors* a_j *and edges* E_t *fixed, then* $\mathcal{F}_{t+1}^{\text{DGM}} = \mathcal{F}_t^{\text{DGM}}$*. If a refinement adds a new anchor and new edge generators, then the updated information structure is the join of* $\mathcal{F}_t^{\text{DGM}}$ *with the new anchor-order events and the new edge generators.*

Corollary 3.11 (Finite number of accepted refinements)**.** *If there is a common nonnegative potential* $\Phi(\mathcal{S})$ *and every accepted refinement decreases* Φ *by at least* $\tau > 0$*, then the number of accepted refinements is at most* $\Phi(\mathcal{S}_0)/\tau$*. In particular, this applies to a fixed scoring buffer or validation objective. The local-buffer descent theorem alone should not be read as a global finiteness result when the scoring buffer changes over time.*

3.5 Canonical local memory

The local memory M_j is the repair mechanism inside an already chosen concept. To keep the main method fixed, the canonical language-model memory is an embedding mean. Other local memories, including ridge and projection-style repair, are implementation extensions in Appendix D.

For classification or language modeling with output embeddings $E[y] \in \mathbb{R}^{d_e}$, concept j stores

$$r_j = \frac{1}{n_j} \sum_{s \in C_j} E[y_s].$$

It contributes the score

$$\ell_{\mathrm{DGM}}(y \mid h) = \sum_{j \in N(h)} \alpha_j(h) E[y]^\top r_j.$$

Repair is the sufficient-statistic update, applied for every selected $j \in N(h)$ with all unselected memories unchanged:

$$r_j^+ = \frac{n_j r_j + \alpha_j(h) E[y]}{n_j + \alpha_j(h)}, \qquad n_j^+ = n_j + \alpha_j(h).$$

The low-cost hard-update variant is recovered by setting $\alpha_{j^\star} = 1$ for the highest-weight concept and $\alpha_j = 0$ for all $j \neq j^\star$.

3.6 From DGM to DGM-Embedding

The generic DGM object above can be attached to many local memories. For sequence models, the canonical instantiation is DGM-Embedding: hidden-state concepts store next-token embedding memories, the read path produces a sparse logit correction, and the observe rule runs after the true next token is revealed. Section 4 gives this layer directly, including the read-before-observe protocol and the forward-only detached write path.

4 DGM-Embedding: A Runtime Memory Layer for Language Models

For language modeling, the world state is a prefix and the consequence is the next observed token. This makes the refine-or-repair decision concrete. If a prefix type is already represented, the layer should repair its next-token memory. If the same runtime concept is being asked to predict incompatible next-token consequences, the layer should refine the concept graph. DGM-Embedding is the canonical large-model layer because its detached target is directly observed: the next-token embedding.

The layer should be read as the language-model analogue of the finite DGM loop. The prefix hidden state chooses edge-gated concepts; their memories produce a sparse logit correction; the true next token then supplies the detached consequence used for repair or refinement. There is no inference-time inner optimization problem in this write path.

4.1 Hidden-state concepts

Let a frozen or slowly trained backbone produce hidden states

$$h_t = \phi_\theta(x_{\leq t}),$$

and let

$$q_t = Q h_t \in \mathbb{R}^{d_q}$$

be the DGM query. A DGM layer maintains frozen query-space anchors $a_j \in \mathbb{R}^{d_q}$, movable query-space centroids $c_j \in \mathbb{R}^{d_q}$, embedding memories $r_j \in \mathbb{R}^{d_e}$, counts, and distinction edges:

$$\mathcal{S}_t^{\mathrm{emb}} = \{a_j, c_j, r_j, n_j\}_{j=1}^{N_t} \cup G_t.$$

The formal information structure is generated by anchors and edges. Retrieval uses centroids as adaptive statistics, while edges are computed from anchors. The recall-oriented candidate set $N(q_t)$ is the same one used by DGM: centroid neighbors for adaptive locality, anchor neighbors for formal route recall, and graph neighbors for recent conflicts.

4.2 Logit correction

The read path is differentiable through the selected memories, routing weights, and backbone parameters. With output embedding $E[y]$, the DGM read contributes a sparse logit correction

$$\Delta\ell_t(y) = \sum_{j \in N(q_t)} \alpha_j(q_t) E[y]^\top r_j,$$

where α_j is edge-gated as in (8). Equivalently, define the mixed memory vector

$$r_{\text{mix}}(q_t) = \sum_{j \in N(q_t)} \alpha_j(q_t) r_j.$$

Then $\Delta\ell_t(y) = E[y]^\top r_{\text{mix}}(q_t)$, and the final logits are

$$\ell(y \mid x_{\leq t}) = \ell_\theta(y \mid x_{\leq t}) + \lambda_t \Delta\ell_t(y).$$

Here λ_t is a learned or fixed trust weight. With tied output embeddings and $d_e = d$, the same correction can be implemented as

$$h_t^{\text{DGM}} = h_t + \lambda_t r_{\text{mix}}(q_t),$$

followed by the usual LM-head multiplication. This avoids an extra vocabulary-sized projection.

4.3 Read before observe

DGM-Embedding uses the same streaming program during training and inference:

$$\text{predict } x_{t+1} \to \text{observe } x_{t+1} \to \text{repair or refine} \to \text{predict } x_{t+2}.$$

The order is important: the layer must predict before it sees the token that will update memory. After the next token is observed, $v_t = E[x_{t+1}]$ gives the detached sufficient-statistic write for every selected $j \in N(q_t)$, with all unselected memories unchanged:

$$r_j^+ = \frac{n_j r_j + \alpha_j(q_t) v_t}{n_j + \alpha_j(q_t)}, \qquad n_j^+ = n_j + \alpha_j(q_t).$$

Hard-update mode is the special case $\alpha_{j^\star} = 1$ and $\alpha_j = 0$ for $j \neq j^\star$. If the penalized refinement test in (12) accepts a new concept, the layer creates a new anchor and centroid at q_t, initializes $r_{\text{new}} = v_t$, and adds distinction edges to $\text{Confused}_q(q_t, x_{t+1})$. Ordinary repairs may update centroids and memories but leave anchors fixed.

During streaming prediction, document continuation, user correction, tool feedback, or supervised next-token evaluation, the next observed token is genuine evidence and can refine or repair the runtime memory. During unconstrained generation, the model should not update from its own sampled token by default, because self-generated samples are actions, not external consequences.

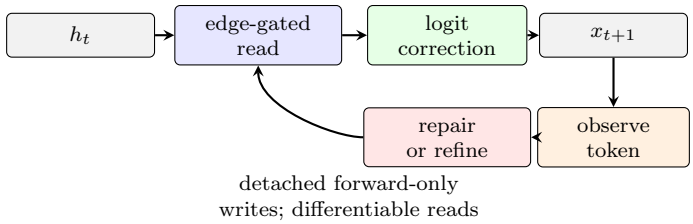

Figure 3: DGM-Embedding sequence update. The layer reads before the next token is known, then uses the observed next token as evidence for repair or refinement.

Algorithm 4 ObserveNextToken for DGM-Embedding

Input: prefix $x_{\leq t}$, observed token x_{t+1}, DGM state $\mathcal{S}_t$
$h_t \leftarrow \phi_\theta(x_{\leq t})$ and $q_t \leftarrow Qh_t$
retrieve $N(q_t)$ and compute edge-gated logits $\ell_\theta + \lambda_t \Delta\ell_t$
$j^\star \leftarrow$ highest-weight concept
$\rho \leftarrow -\log p(x_{t+1} \mid x_{\leq t})$
if $\rho \leq \tau_{\text{repair}}$ **then**
    update DGM-Embedding memories for all $j \in N(q_t)$ using soft credit $\alpha_j(q_t)$
else
    accept a new anchored concept only if the penalized refinement gain test holds
end if
Hard variant: set $\alpha_{j^\star} = 1$ and $\alpha_j = 0$ for $j \neq j^\star$
return updated state

4.4 Sequential detached training

The update is detached:

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

In sequential detached training, the model trains slow parameters through the read path and learns fast state through the same observation rule used at inference. The layer is not a replacement for backpropagation through the backbone. It is a replacement for inference-time and history-time backpropagation through the adaptive state.

Algorithm 5 SequentialDetachedTrainStep for a DGM-Embedding Transformer layer

Input: token block $x_{1:L}$, slow parameters Θ, DGM state $\mathcal{S}$
$\mathcal{L} \leftarrow 0$
for $t = 1, \ldots, L$ **do**
  compute hidden state h_t using the backbone
  $q_t \leftarrow Qh_t$
  retrieve $N(q_t)$ from the concept bank
  compute edge-gated weights $\alpha_j(q_t)$
  apply the DGM-Embedding read
  produce logits for x_{t+1} and accumulate $\mathcal{L} \leftarrow \mathcal{L} + \ell_t$
  $\mathcal{S} \leftarrow \text{stopgrad}(\text{DGMObserve}(\text{stopgrad}(\mathcal{S}), h_t, x_{t+1}))$
end for
update slow parameters Θ by backpropagation through the accumulated selected read computations
return updated Θ and $\mathcal{S}$

Block-lagged approximation. For throughput, an implementation may instead hold $\mathcal{S}$ fixed during the forward pass for a whole block, backpropagate through that block, and then apply detached DGMObserve updates for $t = 1, \ldots, L$. This block-lagged DGM training is more parallel, but it is an approximation: token $t + 1$ in the block does not see the state update produced after token t. It therefore introduces a train/inference mismatch and should not be described as the same observation program as streaming inference.

The main architectural claim stops here: DGM-Embedding is a next-token memory layer with differentiable reads and detached writes. Training modes for representation drift, the optional DGM-Adapter variant, systems notes, and closed-form ridge repair are implementation extensions rather than the paper's central object; we give them in Appendix D.

5 Guarantees

The guarantees are intentionally narrow. They do not claim that DGM is globally optimal, or that a deep backbone can always find the right representation. They formalize the paper's decision rule: decompose runtime excess risk into repair and refinement gaps, repair only inside the current information structure, refine when the current information identifies states with incompatible consequences, and accept DGM updates only when they decrease a local penalized scoring objective. Full proof details are in Appendices I, J, and K.

1. Error decomposes into two gaps. Theorem 2.1 states that for current predictor f_t, current information $\mathcal{F}_t$, and a candidate refinement $\mathcal{G} \supseteq \mathcal{F}_t$,

$$R_L(f_t) - R_L(\mathcal{G}) = \text{RepGap}_L(f_t, \mathcal{F}_t) + \text{RefGap}_L(\mathcal{F}_t, \mathcal{G}).$$

The first term is the best possible gain from repairing the predictor at fixed information. The second is the value of changing the information itself.

2. Missing distinctions obstruct repair. Corollary 2.2 says that if two states have the same information-map value but require different consequences, no predictor measurable with respect to

the current information can satisfy both. This is the main theoretical point of the paper:

if the target separates states that the information map identifies, repair is impossible.

The result is architecture-independent; it applies to caches, adapters, local memories, and test-time optimizers whenever their update leaves the relevant information structure fixed.

3. Refinement has a canonical minimal form. Theorems 2.4 and 2.5 identify the least information update that makes new evidence observable. In the finite case, a binary distinction splits exactly the atoms it crosses. In the sigma-algebra case, the update is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(Z_t).$$

Appendix F gives the standard value identities that explain when such a distinction is useful: conditional mutual information under log loss and conditional variance reduction under squared loss.

4. Guarded DGM updates descend locally. Theorem 3.7 states the algorithmic guarantee. A guarded repair does not increase the scoring-buffer objective because anchors and edges remain fixed and only a non-worsening local-state candidate is committed. An accepted refinement adds one concept and q edges only when its held-out penalized refinement gain beats the repair gain plus margin and statistical safety term. Thus DGM is not an unscored growth heuristic: it is a local refine-or-repair decision with an explicit gap comparison.

These statements also define the boundary of the claim. The scoring buffer is local and changes over time, so the descent result is not a global training theorem. Approximate retrieval must satisfy candidate recall for edge gates to recover the formal hard route. Budgeting, merging, and eviction are systems approximations that may discard distinctions and must be measured separately.

6 Experiments: When Refinement Matters

The experiments are designed to test the paper's central claim rather than broad benchmark performance. A useful task should contain both a repair gap, where local memory updates should help, and a refinement gap, where repair-only methods should collapse incompatible consequences into one state. All learners use a prequential protocol: predict first, observe the consequence, then update. Implementation details and additional sanity checks are in Appendix E.

6.1 Hidden-regime sequence prediction

The most direct check is a hidden-regime online sequence task. A hidden regime chooses the next-token transition map. The learner is not given the regime label; it observes only a noisy continuous cue and the current token, predicts the next token, and then receives the next-token consequence. The same current token can therefore require different next tokens in different regimes. A repair-only memory averages those consequences, while a refine+repair learner can create separate runtime concepts.

Held-out accuracies for the same runs were 0.285 ± 0.049 for cache LM, 0.346 ± 0.106 for budgeted kNN-LM, 0.235 ± 0.024 for nearest-centroid memory, 0.055 ± 0.015 for repair-only DGM, 0.680 ± 0.065 for refine+repair DGM, and 0.282 ± 0.086 for online backpropagation. The important comparison is not only that DGM is higher, but that the repair-only DGM is much worse: the task is constructed so that one local memory is the wrong information structure.

Model	Runtime state	Preq. acc.	Preq. log loss	Concepts
Cache LM	token counts	0.362 ± 0.029	1.443 ± 0.069	–
Budgeted kNN-LM	64 examples	0.583 ± 0.010	4.342 ± 0.338	64 examples
Nearest-centroid memory	16 centroids	0.278 ± 0.035	2.070 ± 0.112	16
DGM repair only	one local memory	0.079 ± 0.017	2.798 ± 0.006	1
DGM refine+repair	graph memory	$\mathbf{0.617 \pm 0.040}$	5.348 ± 0.625	87.6
Backprop MLP, online	one backprop step	0.292 ± 0.020	2.217 ± 0.054	–

Table 2: Hidden-regime online sequence prediction, averaged over five seeds. The useful operation is to create distinctions when identical tokens have regime-dependent consequences. The refine+repair run creates runtime concepts and had monotone local scoring-objective decrease at accepted refinements in all runs. Accuracy and log loss are both reported because the synthetic baselines differ in calibration.

Model	Accepted splits	Prequential accuracy	Test accuracy
Repair only	0	0.491	0.501
Refine+repair	9	0.939	0.967

Table 3: Single-run hard-routing ablation on XOR Gaussian clusters. Local repair alone cannot solve a missing-distinction task, while accepted refinements decrease the local penalized objective and recover the required partition.

6.2 Repair-only ablation

We also run a hard-routing special case on XOR Gaussian clusters. The repair-only model has one leaf and cannot represent XOR, while refine+repair creates accepted distinctions with objective decrease. This is the smallest test of the obstruction theorem: if incompatible labels lie inside one information cell, local repair has no representation in which to store the answer.

6.3 Controlled contextual XOR

As a graph-memory check, we use a two-context XOR stream. In the first context the label is XOR of two Gaussian cluster signs; in the second context the labels are flipped. The context coordinate is observable, but the learner must discover how it changes the consequence. The offline MLP is a batch upper bound; the online MLP predicts each example and then performs one backpropagation step; DGM uses the same online protocol but can refine the concept graph before repairing local state.

6.4 Synthetic next-token stream

Finally, we include a synthetic language-modeling stream. Each document context induces a different next-token transition map. The learner observes a prefix representation and must predict the next token before seeing it. After the next token is revealed, DGM memory stores the prefix type and next-token consequence; online backpropagation again receives one gradient step per token.

These experiments do not claim that DGM is a finished competitive language-modeling system. They check the mechanism that the theory predicts: when the error is caused by a missing distinction, methods that only repair state are forced to mix incompatible consequences; a refine+repair learner can create the missing runtime concept and then repair it locally.

Model	Runtime update	Prequential accuracy	Test accuracy
Backprop MLP, offline	batch backprop	–	0.993 ± 0.002
Backprop MLP, online	one backprop step	0.569 ± 0.065	0.654 ± 0.038
DGM graph memory, online	refine-or-repair	0.804 ± 0.025	0.944 ± 0.026

Table 4: DGM graph memory versus backpropagation on a limited online contextual-XOR stream. Offline backprop is a batch upper bound. The relevant runtime comparison is between online backpropagation and online distinction refinement. The DGM run has monotone local scoring-objective decrease at accepted refinements in all runs.

Task	Offline backprop	Online backprop	Online DGM memory
Synthetic LM	0.931 ± 0.071	0.125 ± 0.021	0.912 ± 0.077

Table 5: Synthetic next-token stream. The task isolates runtime adaptation: the hidden context determines the token transition rule, so the useful operation is to remember a local prefix-to-token consequence after observing it. DGM memory approaches the offline batch MLP while one-step online backpropagation adapts slowly.

7 Related Work

The relevant comparison is not whether DGM uses centers, memory, routing, or online updates. All of these ingredients have long histories. The comparison is whether a runtime learner separates two operations that are often collapsed: repairing state inside the current representation, and refining the representation when consequences reveal a missing distinction.

Repair without refinement. Cache language models and kNN-LMs store or retrieve context-token examples at inference time [GCJ17, KLJ+20]. Fast weights, linear attention memories, passive-aggressive updates, online ridge/RLS, and projection-style memories repair associative state inside a fixed geometry [BHM+16, KVPF20, SIS21, CDK+06, Vov97, Bre67, TA77, HK70]. Test-time adaptation and test-time training update model state or parameters by optimizing an inference-time surrogate [SWL+20, WSL+21, NWZ+22, ZLF22, SLD+25]. These methods address the repair-gap half of runtime learning; DGM asks the prior structural question of whether the error is repairable under the current distinctions.

Refinement without local repair. Decision trees, online trees, Hoeffding trees, adaptive trees, and Mondrian forests refine leaves using information gain, variance reduction, or streaming split tests [BFOS84, DH00, BG09, MWS18, LRT14]. Adaptive Resonance Theory and prototype methods create or move categories under mismatch [CG87b, CG87a, Gro13, Koh90, Fri95, SSZ17]. DGM shares the refinement motivation, but stores distinctions as local graph edges between runtime concepts and attaches repairable memories to the concepts after they are created.

Routing without runtime concept creation. Mixture-of-experts, Switch Transformers, and Product-Key Memory route queries to sparse trained capacity [SMM+17, FZS22, LSR+19]. Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn external memory controllers [WCB15, SWF+15, GWD14, GWR+16]. DGM differs in write semantics: the routed

objects are runtime-created concepts whose edges generate an information structure and whose memories are repaired locally.

Appendix C gives the extended related-work discussion. The short version is that DGM combines refinement and repair but keeps their roles separate:

$$\text{missing consequence distinction} \Rightarrow \text{refine}, \qquad \text{repair gap} \Rightarrow \text{repair}.$$

8 Conclusion

Runtime learning fails when we repair errors that actually require new distinctions. The repair/refinement gap decomposition makes this precise: some excess risk can be removed by changing the predictor at fixed information, and some can only be removed by changing the information itself. Distinction Graph Machines turn this separation into an explicit runtime learner with anchored concepts, local memories, distinction edges, and a read-before-observe update rule.

The theory explains why this separation matters. Contradictions inside an information cell cannot be solved by optimizing a predictor that still factors through the same information map. They require refinement. Once the local concept is adequate, its state can be repaired by sufficient-statistic, ridge, or projection-memory updates. DGM turns that separation into a concrete runtime state: concepts hold local memories, and distinction edges generate the information structure that gates the read path.

The resulting picture places gradient learning, local repair, and distinction refinement in one hierarchy. Backpropagation moves slow parameters; local memory repairs fast state; DGM adds the missing operation of changing the information structure on which those states are defined. In language modeling, this means that a runtime learner need not backpropagate through the entire model at inference time: it can maintain hidden-state concepts, repair next-token memories, and create new distinctions when the stream reveals a genuinely new context.

References

[Ber85] James O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 2 edition, 1985.

[BFOS84] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[BG09] Albert Bifet and Ricard Gavaldà. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII*, volume 5772 of *Lecture Notes in Computer Science*, pages 249–260. Springer, 2009.

[BHM+16] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[Bil95] Patrick Billingsley. *Probability and Measure*. Wiley, 3 edition, 1995.

[Bla53] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.

[BMH+22] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240, 2022.

[Bre67] Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

[CDK+06] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

[CG87a] Gail A. Carpenter and Stephen Grossberg. ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, 26(23):4919–4930, 1987.

[CG87b] Gail A. Carpenter and Stephen Grossberg. A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, 37(1):54–115, 1987.

[CT06] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.

[DGL96] Luc Devroye, László Györfi, and Gábor Lugosi. *A Probabilistic Theory of Pattern Recognition*. Springer, 1996.

[DH00] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80, 2000.

[Fri95] Bernd Fritzke. A growing neural gas network learns topologies. In *Advances in Neural Information Processing Systems*, volume 7, pages 625–632, 1995.

[FZS22] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[GCJ17] Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

[GR07] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[Gro13] Stephen Grossberg. Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, 37:1–47, 2013.

[GW99] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

[GWD14] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

[GWR+16] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

[HK70] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[Hoe63] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[IM98] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998.

[JDJ19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[Kal02] Olav Kallenberg. *Foundations of Modern Probability*. Springer, 2 edition, 2002.

[KLJ+20] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

[KNT+20] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5338–5348, 2020.

[Koh90] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[Koh95] Teuvo Kohonen. *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, 1995.

[KPR+17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[KVPF20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165, 2020.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[LPR17] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[LRT14] Balaji Lakshminarayanan, Daniel M. Roy, and Yee Whye Teh. Mondrian forests: Efficient online random forests. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[LSR+19] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[MS91] Thomas M. Martinetz and Klaus J. Schulten. A neural-gas network learns topologies. In *Artificial Neural Networks*, pages 397–402, 1991.

[MWS18] Chaitanya Manapragada, Geoffrey I. Webb, and Mahsa Salehi. Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1953–1962, 2018.

[NWZ+22] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shihua Zheng, Peilin Zhao, and Mingkui Tan. Efficient test-time model adaptation without forgetting. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16888–16905, 2022.

[Pen55] Roger Penrose. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 51(3):406–413, 1955.

[RHH+16] Jack W. Rae, Jonathan J. Hunt, Tim Harley, Ivo Danihelka, Andrew Senior, Greg Wayne, Alex Graves, and Timothy Lillicrap. Scaling memory-augmented neural networks with sparse reads and writes. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[RRD+16] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2016.

[SIS21] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

[SLD+25] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[SMM+17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[SSZ17] Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[SWF+15] Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[SWL+20] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[TA77] Andrey N. Tikhonov and Vasiliy Y. Arsenin. *Solutions of Ill-Posed Problems*. Winston, 1977.

[Vov97] Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems*, volume 10, 1997.

[WCB15] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

[WRHS22] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[WSL+21] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

[YYLH18] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations*, 2018.

[ZLF22] Marvin Zhang, Sergey Levine, and Chelsea Finn. MEMO: Test time robustness via adaptation and augmentation. In *Advances in Neural Information Processing Systems*, volume 35, pages 38629–38642, 2022.

Appendix

A Appendix Roadmap

The appendix keeps secondary material out of the main method story. It first states limitations, then gives extended related work, implementation extensions, additional computational checks, proof sketches, the projection-memory bridge, and formal proofs.

- **Part 1.** Appendix B clarifies what the theory proves and what it leaves to modeling, statistics, and experiments.
- **Part 2.** Appendix C gives the extended related-work discussion.
- **Part 3.** Appendix D collects tree special cases, data-structure costs, DGM-Embedding training modes, the optional adapter variant, systems notes, and closed-form local repair.
- **Part 4.** Appendix E reports secondary computational checks.
- **Part 5.** Appendix F gives value-of-distinction identities for standard losses.
- **Part 6.** Appendix G gives proof sketches, and Appendix H gives the Projection Memory and CPM bridge.
- **Part 7.** Appendices I, J, and K give the formal proofs.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local; its refinement rule concerns local concepts and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The DGM method specifies one canonical refinement mechanism, based on concepts and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned concept-creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, and better merge or forgetting policies than the default budget notes in Appendix D. The bridge to Projection Memory and CPM is exact for one-hot finite partition memories. Continuous neural representations need additional modeling choices: when a continuous conflict should create a new concept, how local projection memories should be inherited, and how refined information structures interact with differentiable backbones.

Soft sparse retrieval introduces an additional recall condition. Edge gates can reject incompatible candidates, but they cannot select a hard-compatible concept that was never retrieved. The union candidate set in Section 3 reduces this risk, but approximate retrieval remains an engineering assumption that should be measured in large systems.

The large-model layer should be read as an architectural path, not as a completed scaling theorem. It gives a trainable read path, a detached write path, and a default budget policy with

explicit complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing or stale concept shards. Budget maintenance is also outside the formal refinement theorem: merge, eviction, and compression may discard distinctions, so their effect must be evaluated as a resource-management approximation.

The experiments are computational checks, not a benchmark suite. A complete empirical evaluation should include stronger online and nonparametric baselines, ablations of repair-only, refine-only, edge-free, and edge-gated variants, and tasks where hidden context changes force new distinctions.

C Related Work

The relevant comparison is not whether DGM uses centers, memory, routing, or online updates. All of these ingredients have long histories. The question is whether a runtime learner distinguishes two operations that are often collapsed: repairing state inside the current representation, and refining the representation when consequences reveal a missing distinction.

C.1 Methods that repair without refining

Cache language models and kNN-LMs store or retrieve context-token examples at inference time [GCJ17, KLJ+20]. Memorizing Transformers, RETRO, RAG, and sparse retrieval systems extend this idea with larger or external stores [WRHS22, BMH+22, LPP+20, RHH+16]. These methods make past examples available, but the default response to surprise is more storage or retrieval, not the creation of a new runtime concept forced by consequence conflict.

Fast weights, linear attention memories, delta-rule updates, passive-aggressive learning, online ridge/RLS, and Projection Memory repair associative state inside a fixed geometry [BHM+16, KVPF20, SIS21, CDK+06, Vov97, Bre67, TA77, HK70]. Conservative Projection Memory (CPM) is especially close to our repair side because it detects zero innovation with nonzero residual as a conflict. DGM uses this diagnosis differently: a zero-innovation conflict is evidence that the current information cell is too coarse, so projection-style repair should stop until the information structure is refined.

Test-time adaptation and test-time training update model state or parameters by optimizing an inference-time objective [SWL+20, WSL+21, NWZ+22, ZLF22, SLD+25]. These methods ask what surrogate loss should be optimized at test time. DGM asks a prior structural question: is the error repairable under the current distinctions, or does it reveal a missing distinction?

These methods are not wrong. They solve the repair-gap half of runtime learning. They do not, by themselves, provide an explicit consequence-forced refinement operation.

C.2 Methods that refine without local repair

Decision trees, online trees, Hoeffding trees, adaptive trees, extremely fast decision trees, and Mondrian forests refine leaves using information gain, variance reduction, or streaming split tests [BFOS84, DH00, BG09, MWS18, LRT14]. These are the closest algorithmic relatives of DGM's refinement test. The difference is structural: trees store distinctions in a rooted hierarchy, while DGM stores local pairwise distinction edges between runtime concepts and attaches repairable memories to the resulting nodes.

Adaptive Resonance Theory, including ART 1 for binary patterns and ART 2 for analog input patterns, addresses stable incremental category creation under mismatch and vigilance [CG87b, CG87a, Gro13]. Prototype methods, LVQ, SOMs, neural gas, growing neural gas, and Prototypical

Networks represent regions or classes by stored centers or exemplars [Koh90, Koh95, MS91, Fri95, SSZ17]. DGM shares the vocabulary of category creation and prototypes, but its concepts are operational hidden-state concepts whose edges are consequence-forced binary generators. The node is not only a region; it is a place where local memory can be repaired after the distinction is accepted.

Concept Bottleneck Models and Formal Concept Analysis emphasize explicit concept structures [KNT+20, GW99]. DGM should not be read as claiming human-interpretable concepts by default. Its concepts are runtime concepts: hidden-state anchors, centroids, local memories, and pairwise distinctions created from prediction consequences.

These methods solve the refinement half in various forms. They typically do not pair refinement with graph-gated neural local memories that continue to be repaired after the split.

C.3 Methods that route without runtime concept creation

Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to sparse trained capacity [SMM+17, FZS22, LSR+19]. Memory Networks, End-to-End Memory Networks, Neural Turing Machines, and Differentiable Neural Computers learn external read/write controllers [WCB15, SWF+15, GWD14, GWR+16]. These systems are important precedents for sparse reads and external memory. DGM differs in write semantics: the routed objects are runtime-created concepts whose edges generate an information structure and whose memories are repaired locally.

Continual learning studies forgetting, replay, regularization, and capacity growth across task sequences [KPR+17, LPR17, RRD+16, YYLH18]. DGM is complementary. It is an instance-level runtime data structure for local concept growth and memory repair, not a complete task-sequence continual learning system.

C.4 DGM's position

DGM combines refinement and repair but keeps them distinct:

$$\text{missing consequence distinction} \Rightarrow \text{refine}, \qquad \text{repair gap} \Rightarrow \text{repair}.$$

The measure-theoretic formalism follows standard sigma-algebra and conditional-expectation language [Bil95, Kal02]. The Bayes-risk order is related to Blackwell comparison and statistical decision theory [Bla53, Ber85]. The log-loss and squared-loss value identities are standard information-theoretic and Hilbert-space facts [CT06, GR07]. The contribution is to place these facts inside a runtime-learning model: not every error is a gradient, and some errors are missing distinctions.

D DGM-Embedding Extensions

This appendix collects DGM mechanics and large-model details that are useful for implementations but secondary to the main refine-or-repair argument.

D.1 Trees as a hard-routing special case

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors. Trees are still contained in the view below, but as a constrained hard-routing case.

Proposition D.1 (Trees as generalized hard-DGMs)**.** *Given a binary distinction tree, construct one generalized hard-route DGM concept for each leaf and attach the path distinctions as hard eligibility constraints for that concept. If the generalized hard route selects the unique eligible leaf concept and each concept memory equals the tree leaf predictor, then the generalized DGM and the tree make identical predictions.*

Thus the distinction-graph view contains trees after allowing arbitrary concept-level eligibility constraints. A decision tree is the sparse case in which distinctions are forced into a rooted hierarchy and routing is hard and single-path.

D.2 DGM data-structure cost

Let

$$Q(h) = \sum_{j \in N(h)} |I_t(j)|$$

be the number of incident edge gates evaluated for the selected candidates. With exact nearest-neighbor search, prediction costs

$$O(N_t d + Q(h) + |N(h)|\, d + C_{\text{local}}),$$

where C_{local} is the cost of evaluating the selected local memories. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index [IM98, JDJ19]. Creating an anchor-centroid pair costs $O(d)$ plus the cost of initializing its memory. Adding q distinction edges costs $O(qd)$. If each concept stores an embedding mean in $\mathbb{R}^{d_e}$, runtime memory is

$$O(N_t(2d + d_e) + |E_t|d).$$

Theorem D.2 (DGM data-structure bound)**.** *For exact search over N_t centroids in $\mathbb{R}^d$, one edge-gated DGM prediction costs $O(N_t d + Q + |N|\, d + C_{\text{local}})$, where $N = N(h)$ and $Q = Q(h)$. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d+qd)$. With embedding-mean memories, the runtime state uses $O(N_t(2d+d_e)+|E_t|d)$ memory.*

D.3 Ridge local memory

For few-shot adaptation or local readout repair, a memory stores a small support set $Z \in \mathbb{R}^{n \times d}$ and labels $Y \in \mathbb{R}^{n \times C}$. The closed-form adapter is the dual ridge/Tikhonov solution [TA77, HK70]:

$$\Gamma = (ZZ^\top + \lambda I)^{-1} Y, \qquad \text{logits}(z) = zZ^\top \Gamma.$$

Projection Memory and CPM are advanced repair rules inside M_j after the graph has fixed the relevant local concept; Appendix H treats that relation directly.

D.4 Training modes for representation drift

Anchors, centroids, and edges live in query space. If the encoder or query map changes rapidly while the DGM state persists, old anchors and edges describe stale geometry. Reports should therefore state which training mode is used.

Mode A: frozen encoder. Train the backbone and query map first, then freeze ϕ_θ, Q, and the output embedding used by DGM writes while running DGM runtime learning. This is the cleanest protocol for the first experiments and is the default interpretation of the theory.

Mode B: EMA query encoder. Maintain a slowly moving query encoder

$$\bar{\theta} \leftarrow \beta\bar{\theta} + (1-\beta)\theta,$$

and create anchors, centroids, and edge generators in the query space of $\phi_{\bar{\theta}}$ and $Q_{\bar{\theta}}$. The backbone can still train, but the DGM state is tied to the slower geometry.

Mode C: periodic state rebuild. During end-to-end training, rebuild or clear the DGM state every K optimization steps, or rebuild it from a replay buffer under the current query map. The formal guarantees then apply within a rebuild interval. Cross-interval state carryover is a systems approximation, not a claim of anchor-stable refinement.

D.5 Non-canonical adapter extension

The DGM-Embedding layer is the canonical layer because its target is the observed next-token embedding. A richer optional layer stores a local linear repair map, but it requires an explicit residual target and should not be conflated with the next-token memory mechanism:

$$\mathcal{S}_t^{\text{adp}} = \{a_j, c_j, W_j, n_j\}_{j=1}^{N_t} \cup G_t, \qquad W_j \in \mathbb{R}^{d\times r}.$$

Let $K : \mathbb{R}^d \to \mathbb{R}^r$ be a learned or fixed key projection and set $k_t = Kh_t$. The adapter read is

$$\Delta h_t = \sum_{j\in N(q_t)} \alpha_j(q_t) W_j k_t, \qquad \tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t.$$

Here $W_j k_t$ is a hidden-space residual, so the write rule requires an explicit residual target $v_t \in \mathbb{R}^d$. Natural choices include a token-embedding target $PE[x_{t+1}]$, a teacher-forced output-error target $P(E[x_{t+1}] - \hat{e}_t)$, or a distillation target $h_t^{\text{teacher}} - h_t$. Without such a target, the adapter is only an architectural extension.

Given a local residual target, the projected rank-one repair for a selected concept is the same residual-correction pattern that appears in passive-aggressive updates, RLS/ridge-style online least squares, and delta-rule fast-weight memories [CDK+06, Vov97, SIS21]:

$$W_j^+ = W_j + \frac{\eta\alpha_j(q_t)}{1 + \eta\alpha_j(q_t)\,\|k_t\|_2^2}(v_t - W_j k_t)k_t^\top.$$

Equivalently, a memory may maintain small ridge sufficient statistics and solve the local adapter in closed form [TA77, HK70].

D.6 Cost and systems notes

During sequential detached training, backpropagation sees the selected read computation, while the DGM state is updated in the same order as streaming inference. The important comparison is with test-time gradient adaptation. A TTT-style layer typically performs an inner backward pass at inference time. A DGM layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost is forward-only:

$$\text{DGM inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

Theorem D.3 (DGM layer cost)**.** *Consider a DGM adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|(d_q + dr)).$$

For a block of length L, the DGM state history need not be stored for backpropagation; the additional activation memory is the selected read computation, not the full sequence of memory states. The DGM-Embedding layer replaces the $|N|dr$ adapter term by the selected embedding-logit cost.

The concept bank can be sharded by the same principle used in retrieval and sparse expert systems. Each device owns a subset of concepts. A query is routed to a small set of shards, each shard returns local top-k candidates, and a final top-k merge selects the memories used by the read path. Only selected memories receive gradients. Detached writes are sent back to the owning shards.

Budgeting is necessary for long training. A practical DGM layer maintains $N_t \leq N_{\max}$, merges only compatible unprotected concepts, and otherwise evicts or compresses low-score memories. These budget actions are maintenance operations, not learning operations in the formal refine-or-repair sense. The refine-or-repair theorems apply between maintenance events; merge, eviction, and compression may destroy information and must be evaluated as resource-management approximations.

D.7 Closed-form local repair

Ridge memory is the main closed-form adapter used in our checks. Let $Z \in \mathbb{R}^{n \times d}$ be normalized support embeddings stored by a DGM memory and let $Y \in \mathbb{R}^{n \times C}$ be one-hot consequences. The adapter computes the dual ridge/Tikhonov solution [TA77, HK70]

$$\Gamma = (ZZ^\top + \lambda I)^{-1} Y, \qquad \text{logits}(z) = zZ^\top \Gamma.$$

This is projection-style repair: it fits the local adapter in one linear solve rather than by iterated backpropagation. In a large model, the same idea can be applied per concept, per document, or per retrieved shard after the relevant distinction structure has been fixed.

E Additional Computational Checks

This appendix reports secondary checks that are useful for debugging implementations but are not the main evidence for the refine-or-repair claim.

E.1 Online image streams

We run small online image-stream checks on MNIST and CIFAR-10 using cached local datasets. These are sanity checks, not evidence that DGM is a competitive image classifier. In these experiments the DGM memory uses exemplar repair. The online backpropagation baseline sees each example once and performs one gradient step. Offline backpropagation is a batch upper bound.

The relevant future settings are context switching, few-shot adaptation with a frozen encoder, long-document memory, repeated syntax regimes, and hidden task identity.

Dataset	Offline backprop	Online backprop	Online DGM memory
MNIST	0.843 ± 0.009	0.682 ± 0.038	0.851 ± 0.003
CIFAR-10	0.273 ± 0.003	0.137 ± 0.010	0.243 ± 0.008

Table 6: Image-stream sanity checks. The point is not to beat fully trained image models, but to compare one-pass runtime updates.

E.2 Frozen-embedding adapter check

For a stronger adapter benchmark, we train a shared convolutional encoder and freeze it. The adaptation problem is deliberately narrow: given only five labeled support examples per class, adapt the readout. We compare nearest-neighbor memory, ridge memory as a closed-form projection adapter from Appendix D, a one-pass backpropagation head, and a 200-step backpropagation head on the same frozen embeddings.

Dataset	Method	Accuracy	Adapter time
MNIST, 5-shot	one-pass backprop head	0.100 ± 0.000	0.0002s
MNIST, 5-shot	200-step backprop head	0.332 ± 0.009	0.0320s
MNIST, 5-shot	kNN memory	0.486 ± 0.005	0.0006s
MNIST, 5-shot	ridge memory	$\mathbf{0.623 \pm 0.003}$	0.0018s
CIFAR-10, 5-shot	one-pass backprop head	0.115 ± 0.014	0.0003s
CIFAR-10, 5-shot	200-step backprop head	0.359 ± 0.009	0.0325s
CIFAR-10, 5-shot	kNN memory	0.355 ± 0.005	0.0006s
CIFAR-10, 5-shot	ridge memory	$\mathbf{0.397 \pm 0.010}$	0.0004s

Table 7: Few-shot adapter check on frozen embeddings. Ridge memory fits the adapter by a closed-form projection solve.

E.3 Projection-memory residual identity

We verify the projection-memory residual contraction identity used by local repair. For random W, k, v and step size η, the soft Projection Memory update satisfies

$$v - W_{t+1}k = \frac{1}{1 + \eta \|k\|_2^2}(v - W_t k).$$

Over 200 random trials, the maximum absolute error between the measured residual ratio and the formula was $3.33 \cdot 10^{-16}$.

F The Value of a Distinction

Minimal refinement says how to add a distinction; it does not license every possible distinction. The value question is task-dependent: when does the new distinction actually change the best achievable prediction? We measure this by the decrease in optimal risk after the information structure is refined, in the same decision-theoretic sense used in classical comparisons of statistical experiments and value of information [Bla53, Ber85].

F.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$-measurable predictor is also $\mathcal{G}$-measurable. The decision class has only grown; this is the Bayes-risk analogue of Blackwell's monotonicity of decision value under more informative experiments [Bla53].

Theorem F.1 (Bayes-risk monotonicity)**.** *If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L,*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

This is a population statement. It says that information cannot hurt the best possible decision rule. It does not say that a finite implementation should refine without restraint. Statistical estimation, finite samples, and model selection can overfit; those are algorithmic questions, not failures of the information order. This is why the DGM algorithm later uses penalties, thresholds, and sample-split scoring before committing a new concept.

F.2 Log loss

For a finite target variable Y, log loss asks the learner to output a conditional distribution. The optimal prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy, a standard identity for logarithmic scoring rules and information theory [CT06, GR07].

Theorem F.2 (Log-loss value)**.** *Let Y be finite and let the loss be $-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Thus, for classification and language modeling, a refinement is useful exactly to the extent that it carries conditional mutual information about the consequence beyond what the learner already knows. If the new bit does not tell us anything about the next consequence beyond the old information, it is a gratuitous distinction, even if it separates inputs.

F.3 Squared loss

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y \mid \mathcal{F}]$, the L^2 projection onto the closed subspace of $\mathcal{F}$-measurable random variables [Bil95, Kal02]. Refinement helps precisely when it changes the conditional mean.

Theorem F.3 (Squared-loss value)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value when the cells it creates have different conditional means. In regression language, refinement is worth paying for when it separates consequences, not merely when it separates inputs.

F.4 Finite empirical scores

The population identities lead directly to standard empirical split scores: information gain for classification and variance reduction for regression trees [BFOS84, DH00]. Suppose an atom B contains samples (x_j, y_j) and a candidate distinction $g : B \to \{0, 1\}$ splits it into B_0 and B_1.

For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss improvement is

$$\Delta_{\log}(B, g) = |B|H(\hat{p}_B) - |B_0|H(\hat{p}_{B_0}) - |B_1|H(\hat{p}_{B_1}).$$

Equivalently,

$$\Delta_{\log}(B, g) = |B|\widehat{I}(Y; g \mid B).$$

For regression, define

$$\hat{\mu}_B = \frac{1}{|B|}\sum_{j\in B} y_j, \qquad \mathrm{SSE}(B) = \sum_{j\in B} \|y_j - \hat{\mu}_B\|_2^2 .$$

The empirical squared-loss improvement is

$$\Delta_2(B, g) = \mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1).$$

It also has the between-cell form

$$\Delta_2(B, g) = |B_0|\,\|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\,\|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2 .$$

These scores implement the same principle as the population theory: split only when the proposed distinction has measurable consequence value. The DGM algorithm below turns this population rule into a guarded empirical decision.

G Proof Sketches

This section collects the main proof ideas in the order used by the paper: runtime excess risk decomposes into repair and refinement gaps; contradictions require distinctions; minimal refinement adds the missing distinction and nothing else; useful distinctions have exact value under standard losses; and guarded DGM updates decrease the local penalized objective. The measure-theoretic notation, conditional-expectation facts, and decision-theoretic Bayes-risk framing are standard [Bil95, Kal02, Bla53, Ber85]. Full formal versions appear in Appendices I, J, and K.

Theorem G.1 (Runtime error decomposition, proof sketch for Theorem 2.1)**.** *The statement is Theorem 2.1.*

Proof sketch. Add and subtract the Bayes risk under the current information:

$$R_L(f_t) - R_L(\mathcal{G}) = \big(R_L(f_t) - R_L(\mathcal{F}_t)\big) + \big(R_L(\mathcal{F}_t) - R_L(\mathcal{G})\big).$$

The first term is nonnegative because f_t is one of the $\mathcal{F}_t$-measurable predictors over which $R_L(\mathcal{F}_t)$ optimizes. The second is nonnegative because $\mathcal{G} \supseteq \mathcal{F}_t$ enlarges the admissible predictor class. □

Theorem G.2 (Contradiction, proof sketch for Theorem I.3)**.** *The statement is Corollary 2.2.*

Proof sketch. If f is $\mathcal{P}$-measurable and ω, ω' lie in the same atom, then

$$f(\omega) = f(\omega').$$

If $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$, then

$$y(\omega) = y(\omega'),$$

contradicting the hypothesis. □

Theorem G.3 (Minimal finite refinement, proof sketch for Theorem I.1)**.** *The statement is Theorem 2.4.*

Proof sketch. Every atom of $\mathcal{P}_E$ lies inside an atom of $\mathcal{P}$, so $\mathcal{P}_E$ refines $\mathcal{P}$. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

so E is $\mathcal{P}_E$-measurable. If $\mathcal{Q}$ refines $\mathcal{P}$ and makes E measurable, then every atom $Q \in \mathcal{Q}$ satisfies

$$Q \subseteq B$$

for some $B \in \mathcal{P}$, and also either $Q \subseteq E$ or $Q \subseteq E^c$. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus $\mathcal{Q}$ refines $\mathcal{P}_E$. □

Theorem G.4 (Minimal sigma-algebra refinement, proof sketch for Theorem I.2)**.** *The statement is Theorem 2.5.*

Proof sketch. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

This sigma-algebra contains $\mathcal{F}$ and makes Z measurable. If $\mathcal{G}$ also contains $\mathcal{F}$ and makes Z measurable, then

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

□

Theorem G.5 (Risk monotonicity, proof sketch for Theorem J.1)**.** *The statement is Theorem F.1.*

Proof sketch. If $\mathcal{F} \subseteq \mathcal{G}$, then every $\mathcal{F}$-measurable action is $\mathcal{G}$-measurable. Therefore

$$\{a : a \text{ is } \mathcal{F}\text{-measurable}\} \subseteq \{a : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Taking the infimum of the same expected loss over the larger set gives

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

□

Theorem G.6 (Log-loss value, proof sketch for Theorem J.2)**.** *The statement is Theorem F.2.*

Proof sketch. For any $\mathcal{F}$-measurable conditional distribution q, the standard log-loss decomposition gives

$$\mathbb{E}[-\log q(Y)] = H(Y \mid \mathcal{F}) + \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}) \| q(\cdot))\right].$$

The minimizer is $q^\star = \mathbb{P}(Y \in \cdot \mid \mathcal{F})$, so $R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F})$. Similarly $R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G})$. Therefore

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

□

Theorem G.7 (Squared-loss value, proof sketch for Theorem J.3)**.** *The statement is Theorem F.3.*

Proof sketch. The optimal $\mathcal{F}$-predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, the standard L^2 projection. Since $\mathcal{F} \subseteq \mathcal{G}$,

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = (Y - \mathbb{E}[Y \mid \mathcal{G}]) + (\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]) .$$

The two terms are orthogonal because the first has conditional mean zero given $\mathcal{G}$ and the second is $\mathcal{G}$-measurable. Taking squared ℓ_2 norms and expectations gives the identity. □

Theorem G.8 (Anchor edge separation, proof sketch for Theorem K.1)**.** *The statement is Theorem 3.1.*

Proof sketch. With $u = (a_i - a_j)/\left\|a_i - a_j\right\|_2$ and $b = u^\top(a_i + a_j)/2$,

$$u^\top a_i - b = \frac{1}{2}u^\top(a_i - a_j) = \frac{1}{2}\left\|a_i - a_j\right\|_2 ,$$
$$u^\top a_j - b = -\frac{1}{2}u^\top(a_i - a_j) = -\frac{1}{2}\left\|a_i - a_j\right\|_2 .$$

□

Theorem G.9 (DGM graph partition, proof sketch for Theorem K.2)**.** *The statement is Theorem 3.2.*

Proof sketch. The graph signature records the value of every binary edge generator. Equality of signatures is an equivalence relation, and its classes are exactly the nonempty common level sets of those generators. Adding a new edge appends one binary coordinate to the signature, so each old atom is either left intact or cut by the two events $g^{-1}(1)$ and $g^{-1}(0)$. □

Theorem G.10 (Hard-route measurability, proof sketch for Theorem K.3)**.** *The statement is Theorem 3.3.*

Proof sketch. Edge eligibility is determined by finitely many edge-bit events, so it is measurable with respect to $\mathcal{F}_t^{\text{edge}}$. The anchor-order sigma-algebra generated by all pairwise anchor comparisons determines the nearest anchor in any eligible subset under deterministic tie-breaking. The exact hard selected index is a finite composition of these two finite decisions, so it is measurable with respect to their join. Movable centroids can still be used for the soft retrieval path, but they are not part of this formal information structure. □

Theorem G.11 (Soft routing hard limit, proof sketch for Theorem K.4)**.** *The statement is Theorem 3.5.*

Proof sketch. The sparse soft read can only assign mass to candidates it retrieved. Therefore, if the exact hard-route concept is omitted from the candidate set, no temperature limit can recover that route. Under candidate recall, the remaining argument is a finite softmax and gate-sharpening calculation. For ordinary nearest-center routing, every nonnearest candidate has unnormalized weight smaller by at least $\exp(-\delta/T)$, so the total nonnearest mass is bounded by a finite candidate-count factor times this exponential. For a structural edge gate with uniformly positive weight, a wrong-side signed margin $-\delta_e$ contributes a log-sigmoid term bounded by $-\delta_e/\tau_e$. Degree normalization rescales the exponent but does not remove exponential decay under a uniform degree bound. After normalizing against an eligible candidate with compatibility bounded below and bounded distance-score ratio, the wrong-side mass vanishes as the edge temperature tends to zero. □

Theorem G.12 (Refine-or-repair descent, proof sketch for Theorem K.5)**.** *The statement is Theorem 3.7.*

Proof sketch. Guarded repair proposes local memory or centroid changes while keeping anchors and edges fixed, then commits only candidates that do not increase empirical loss on the fixed scoring buffer $\mathcal{B}_t^{\text{score}}$. The identity candidate is always feasible. The concept and edge penalties therefore stay fixed, so the local objective $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. A refinement that creates one anchored concept and q edges changes $J_{\mathcal{B}_t^{\text{score}}}$ by the empirical-risk decrease on the scoring buffer minus $\lambda_{\text{concept}} + \lambda_{\text{edge}} q$, which is exactly the penalized refinement gain in (11). The acceptance test requires this net decrease to exceed the empirical repair gain plus $\tau + 2\epsilon_m$. □

Theorem G.13 (Safe refinement, proof sketch for Theorem K.6)**.** *The statement is Theorem 3.8.*

Proof sketch. Conditioning on a finite candidate class fixed before scoring, or generated from a proposal buffer independent of the scoring buffer, Hoeffding's inequality bounds the deviation between empirical and population risk for the old state and for each candidate-refined state [Hoe63]. A union bound over this finite collection gives the usual finite-class validation guarantee [DGL96]. Each gain is a difference of two risks, so its empirical-population deviation is at most twice the risk deviation. If the empirical gain exceeds cost plus $\tau + 2\epsilon_m$, the population gain exceeds cost plus τ on this high-probability event. □

Corollary G.14 (Bounded LM score, proof sketch for Corollary K.7)**.** *The statement is Corollary 3.9.*

Proof sketch. Clipping and normalizing next-token negative log likelihood gives a score in $[0, 1]$. The finite-class refinement theorem therefore applies directly to the clipped scoring rule. The conclusion is intentionally about clipped population gain, not raw unbounded cross-entropy. □

Theorem G.15 (Anchor-stable DGM information structure, proof sketch for Theorem K.8)**.** *The statement is Theorem 3.10.*

Proof sketch. Every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is a deterministic function of the two frozen endpoint anchors. A repair step changes neither endpoint anchor, so it changes neither u_e nor b_e. It also leaves every anchor-distance comparison event unchanged. Therefore the anchor-generated DGM information structure is unchanged, even though the retrieval centroids and local memories may be repaired. □

Corollary G.16 (Finite refinements, proof sketch for Corollary K.9)**.** *The statement is Corollary 3.11.*

Proof sketch. After m accepted refinements a common potential Φ has decreased by at least $m\tau$. Since the potential is nonnegative, $m\tau \leq \Phi(\mathcal{S}_0)$. This argument requires a common potential; changing local scoring buffers do not by themselves give a global finiteness bound. □

Proposition G.17 (Tree as generalized hard-DGM, proof sketch for Proposition K.10)**.** *The statement is Proposition D.1.*

Proof sketch. Make one concept for each tree leaf and attach the root-to-leaf binary tests as hard eligibility constraints. Every input satisfies exactly the path constraints of the leaf reached by the tree, so the generalized hard route selects the same memory and returns the same prediction. □

Theorem G.18 (DGM data structure, proof sketch for Theorem K.11)**.** *The statement is Theorem D.2.*

Proof sketch. A prediction computes centroid and anchor distances from h to N_t concepts in $\mathbb{R}^d$, costing

$$O(N_t d).$$

It then evaluates $Q(h)$ selected incident edge gates and $|N(h)|$ selected local memories, costing $O(Q(h) + |N(h)|d + C_{\text{local}})$ including sparse routing weights. Guarded repair adds the cost of scoring proposed local candidates. Creating one anchor-centroid pair stores two vectors in $\mathbb{R}^d$, still $O(d)$. Adding q distinction edges stores q vectors and thresholds, costing

$$O(qd).$$

For mean memories, each concept stores $a_j, c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, while each edge stores one distinction vector in $\mathbb{R}^d$. Hence memory is

$$O(N_t(2d + d_e) + |E_t|d).$$

□

Theorem G.19 (DGM layer cost, proof sketch for Theorem K.12)**.** *The statement is Theorem D.3.*

Proof sketch. Computing $q = Qh$ costs $O(dd_q)$. Retrieval costs $C_{\text{ret}}(N_t, d_q)$ by definition. Edge-gate evaluation over selected incident edges costs $O(Q_N)$. For each selected concept, a rank-r adapter computes

$$W_j K h.$$

The matrix-vector product with one local adapter costs $O(dr)$, so $|N|$ selected adapters cost

$$O(|N|dr).$$

The detached write path repeats query, retrieval, and edge-gate evaluation, then updates only selected local states. A rank-one local repair or low-rank sufficient-statistic update costs $O(dr)$ per selected memory, with $O(d_q)$ additional concept bookkeeping. Hence write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|(d_q + dr)).$$

Because the write path is detached, backpropagation does not need intermediate copies of $\mathcal{S}_t$ for all t. It stores the selected read computation used in the forward loss. □

Theorem G.20 (One-hot memory, proof sketch for Theorem K.13)**.** *The statement is Theorem H.1.*

Proof sketch. If $\omega, \omega' \in B_i$, then both one-hot features equal e_i, so

$$W\phi_{\mathcal{P}}(\omega) = We_i = W\phi_{\mathcal{P}}(\omega').$$

Thus $W\phi_{\mathcal{P}}$ is constant on atoms. Conversely, if f is constant on each B_i, put that constant value in column i of W. Then

$$f(\omega) = W\phi_{\mathcal{P}}(\omega)$$

for every ω. □

Theorem G.21 (CPM conflict, proof sketch for Theorem K.14)**.** *The statement is Theorem H.2.*

Proof sketch. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

After committing $We_i = v(\omega)$, the later evidence $We_i = v(\omega')$ has the same key. Its innovation relative to the old key span is

$$z = (I - e_i e_i^\top)e_i = 0.$$

Its residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega) \neq 0.$$

□

H From Distinctions to Projection Memory

DGM uses refinement and repair in the same runtime object. Distinction learning asks what the learner can tell apart. Projection learning assumes such a representation and asks how a state inside it should be minimally repaired. The repair side is related to Bregman projection, passive-aggressive learning, online regression, and fast-weight memory updates [Bre67, CDK+06, Vov97, BHM+16, SIS21]. This section makes the relation formal in the finite one-hot case.

H.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}} : \Omega \to \mathbb{R}^n$ be its one-hot feature map:

$$\phi_{\mathcal{P}}(\omega) = e_i \qquad \text{if} \qquad \omega \in B_i.$$

A memory matrix $W \in \mathbb{R}^{m \times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem H.1 (One-hot partition memory)**.** *For every $W \in \mathbb{R}^{m \times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

H.2 CPM conflict as missing distinction

For old key matrices K, a new key k has CPM innovation

$$z = (I - KK^\dagger)k. \tag{13}$$

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem H.2 (CPM conflict equals missing distinction)**.** *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω'.

H.3 The hierarchy

The resulting hierarchy is

$$\text{distinction refinement} \supset \text{projection repair} \supset \text{implicit gradient} \supset \text{explicit gradient descent.}$$

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection, and backpropagation is the mechanism that computes such gradients in differentiable static graphs [RHW86].

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

I Minimal Refinement Proofs

The statements in this appendix use standard facts about finite partitions, generated sigma-algebras, and measurability [Bil95, Kal02]. The contribution is the runtime-learning interpretation: refinement is the minimal information update needed by new evidence.

Theorem I.1 (Minimal finite refinement, formal version of Theorem 2.4)**.** *Let Ω be finite, let $\mathcal{P}$ be a partition of Ω, and let $E \subseteq \Omega$. Define*

$$\mathcal{P}_E = \{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}.$$

Then:

- **Part 1.** *$\mathcal{P}_E$ is a partition of Ω.*
- **Part 2.** *$\mathcal{P}_E$ refines $\mathcal{P}$.*
- **Part 3.** *E is $\mathcal{P}_E$-measurable.*
- **Part 4.** *If $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

Consequently, $\mathcal{P}_E$ is the unique minimal refinement of $\mathcal{P}$ making E measurable.

Proof. For each $B \in \mathcal{P}$, the nonempty sets among $B \cap E$ and $B \cap E^c$ are disjoint and have union B. Since the atoms of $\mathcal{P}$ are disjoint and cover Ω, the atoms of $\mathcal{P}_E$ are disjoint and cover Ω. Thus Part 1 holds.

Every atom of $\mathcal{P}_E$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

where empty intersections are ignored. Hence E is a union of atoms of $\mathcal{P}_E$, proving Part 3.

Let $\mathcal{Q}$ refine $\mathcal{P}$ and make E measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

Since E is $\mathcal{Q}$-measurable, each atom of $\mathcal{Q}$ is either contained in E or disjoint from E. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_E$, so $\mathcal{Q}$ refines $\mathcal{P}_E$. This proves Part 4.

If another partition has the same minimal property, it refines $\mathcal{P}_E$ and is refined by $\mathcal{P}_E$, so the two partitions have the same atoms. □

Theorem I.2 (Minimal sigma-algebra refinement, formal version of Theorem 2.5)**.** *Let $(\Omega, \mathcal{A})$ be a measurable space, let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra, and let $Z : \Omega \to \mathcal{Z}$ be a measurable map into a measurable space $(\mathcal{Z}, \mathcal{B})$. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{A}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Proof. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(Z)$ and $\sigma(Z) \subseteq \mathcal{F} \vee \sigma(Z)$, so Z is measurable with respect to $\mathcal{F} \vee \sigma(Z)$.

Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable. The latter condition means $\sigma(Z) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

Thus $\mathcal{F} \vee \sigma(Z) \subseteq \mathcal{G}$. □

Theorem I.3 (Contradiction requires distinction, formal version of Corollary 2.2)**.** *Let $I : \Omega \to \mathcal{Z}$ be an information map and let $y : \Omega \to A$ be a required output. Suppose $I(\omega) = I(\omega')$ but $y(\omega) \neq y(\omega')$. Then no predictor $f = g \circ I$ satisfies both*

$$f(\omega) = y(\omega), \qquad f(\omega') = y(\omega').$$

Proof. Since $f = g \circ I$ and $I(\omega) = I(\omega')$,

$$f(\omega) = f(\omega').$$

If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

contradicting $y(\omega) \neq y(\omega')$. □

J Value of Distinction Proofs

The proofs below use standard tools from statistical decision theory, information theory, proper scoring rules, and conditional expectation: Bayes risk under constrained information, log-loss optimality of the conditional law, and the L^2 projection identity for conditional expectation [Bla53, Ber85, CT06, GR07, Bil95, Kal02].

Theorem J.1 (Bayes-risk monotonicity, formal version of Theorem F.1)**.** *Let Y be a target random variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \to A : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \to A : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y, a)] \\ &\leq \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y, a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

□

Theorem J.2 (Log-loss value, formal version of Theorem F.2)**.** *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y, and the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Proof. For any $\mathcal{F}$-measurable conditional distribution q, write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\mathrm{KL}}(p_{\mathcal{F}} \| q). \end{aligned}$$

Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given $\mathcal{F}$,

$$I(Y; \mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

Combining proves the claim. □

Theorem J.3 (Squared-loss value, formal version of Theorem F.3)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

Proof. The optimal $\mathcal{F}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$U = Y - \mathbb{E}[Y \mid \mathcal{G}], \qquad V = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = U + V.$$

Since V is $\mathcal{G}$-measurable and $\mathbb{E}[U \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle U, V\rangle \mid \mathcal{G}] = \langle \mathbb{E}[U \mid \mathcal{G}], V\rangle = 0.$$

Thus

$$\begin{aligned}\mathbb{E}\,\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\,\|U + V\|_2^2 \\ &= \mathbb{E}\,\|U\|_2^2 + \mathbb{E}\,\|V\|_2^2 \\ &= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].\end{aligned}$$

Rearranging proves the identity. □

Proposition J.4 (Finite squared-loss split identity)**.** *Let $B = B_0 \cup B_1$ be a disjoint split of a finite sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and* SSE *as in Section F,*

$$\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1) = |B_0|\,\|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\,\|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

Proof. For each child B_b,

$$\begin{aligned}\sum_{j \in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\ &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b|\,\|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2,\end{aligned}$$

because

$$\sum_{j \in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

Summing over $b \in \{0, 1\}$ gives the result. □

K DGM Algorithm and Projection-Bridge Proofs

This appendix combines elementary finite graph/partition arguments with standard concentration and linear-algebra tools: Hoeffding finite-class validation bounds, nearest-neighbor indexing costs, Moore-Penrose projections, and ridge/Tikhonov least squares [Hoe63, DGL96, IM98, JDJ19, Pen55, TA77, HK70].

Theorem K.1 (Anchor edge separation, formal version of Theorem 3.1)**.** *Let $a_i, a_j \in \mathbb{R}^d$ with $a_i \neq a_j$. Define*

$$u = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b = \frac{1}{2} u^\top (a_i + a_j).$$

Then

$$u^\top a_i - b = \frac{1}{2} \|a_i - a_j\|_2 , \qquad u^\top a_j - b = -\frac{1}{2} \|a_i - a_j\|_2 .$$

Proof. Compute

$$\begin{aligned} u^\top a_i - b &= u^\top a_i - \frac{1}{2} u^\top a_i - \frac{1}{2} u^\top a_j \\ &= \frac{1}{2} u^\top (a_i - a_j) \\ &= \frac{1}{2} \|a_i - a_j\|_2 . \end{aligned}$$

Similarly,

$$\begin{aligned} u^\top a_j - b &= u^\top a_j - \frac{1}{2} u^\top a_i - \frac{1}{2} u^\top a_j \\ &= -\frac{1}{2} u^\top (a_i - a_j) \\ &= -\frac{1}{2} \|a_i - a_j\|_2 . \end{aligned}$$

□

Theorem K.2 (DGM graph generates a finite information structure, formal version of Theorem 3.2)**.** *Let E be a finite set of binary generators $g_e : \mathcal{H} \to \{0, 1\}$. Define*

$$\sigma_E(h) = \big(g_e(h)\big)_{e \in E}.$$

The relation $h \sim_E h'$ iff $\sigma_E(h) = \sigma_E(h')$ is an equivalence relation. Its nonempty equivalence classes form a finite partition $\mathcal{P}_E^G$ of $\mathcal{H}$. If a new generator g is added, then

$$\mathcal{P}_{E \cup \{g\}}^G = \mathcal{P}_E^G \vee \sigma(g),$$

where the right side denotes cutting each atom of $\mathcal{P}_E^G$ by $g^{-1}(1)$ and $g^{-1}(0)$ and keeping nonempty pieces.

Proof. Equality of signatures is reflexive, symmetric, and transitive, so $\sim_E$ is an equivalence relation. Equivalence classes are disjoint, nonempty, and cover $\mathcal{H}$, hence they form a partition. Since E is finite, there are at most $2^{|E|}$ signatures, so the partition is finite.

For the update, two points have the same signature under $E \cup \{g\}$ exactly when they have the same old E-signature and the same value of g. Therefore each new equivalence class is an old atom intersected with either $g^{-1}(1)$ or $g^{-1}(0)$, with empty intersections discarded. This is precisely $\mathcal{P}_E^G \vee \sigma(g)$. □

Theorem K.3 (Anchor-based hard routing is graph-plus-anchor-order measurable, formal version of Theorem 3.3)**.** *Let E_t be finite and let $a_1, \dots, a_{N_t} \in \mathbb{R}^d$ be finite frozen anchors with deterministic tie-breaking. Let $\mathcal{F}_t^{\text{edge}} = \sigma(g_e : e \in E_t)$ and let $\mathcal{F}_t^{\text{anchor-order}}$ be generated by all pairwise anchor-distance comparison events*

$$\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\}.$$

The exact hard-route selected concept index, defined as the nearest eligible anchor with fallback to the nearest anchor, is measurable with respect to $\mathcal{F}_t^{\text{edge}} \vee \mathcal{F}_t^{\text{anchor-order}}$.

Proof. For each concept j, eligibility is an intersection of finitely many edge-bit events of the form

$$\{h : g_e(h) = 1\} \qquad \text{or} \qquad \{h : g_e(h) = 0\},$$

depending on j's recorded side of edge e. Hence the eligible set for j is $\mathcal{F}_t^{\text{edge}}$-measurable.

The anchor-order sigma-algebra determines the complete order of all anchors under the fixed tie-breaking rule. For any subset $A \subseteq \{1, \dots, N_t\}$, the event that concept j is the nearest anchor in A is a finite intersection of pairwise anchor-comparison events, hence belongs to $\mathcal{F}_t^{\text{anchor-order}}$. The selected hard-route index is obtained by combining finitely many eligibility events with these nearest-in-subset events. Therefore the inverse image of each selected index is a finite union of intersections of sets from $\mathcal{F}_t^{\text{edge}}$ and $\mathcal{F}_t^{\text{anchor-order}}$, and belongs to their join. □

Theorem K.4 (Soft routing approaches hard routing, formal version of Theorem 3.5)**.** *Let N be a finite candidate set and let*

$$\alpha_j = \frac{\exp(-d_j/T)\chi_j}{\sum_{r \in N} \exp(-d_r/T)\chi_r}.$$

If $j^\star_{\text{hard}} \notin N$, where $j^\star_{\text{hard}}$ is the exact hard-route selected concept, then this sparse soft read gives no guarantee for the omitted hard-route concept. Otherwise, assume $j^\star = j^\star_{\text{hard}} \in N$. If $j^\star$ is the unique candidate with $d_j - d_{j^\star} \ge \delta > 0$ for all $j \ne j^\star$ and $\chi_j = \chi_{j^\star} = 1$, then

$$1 - \alpha_{j^\star} \le (|N| - 1)\exp(-\delta/T).$$

Moreover, suppose $\log \chi_j$ contains a term

$$\frac{\beta_e}{\sqrt{D_j}} \log \operatorname{sigmoid}\left(\frac{m_j}{\tau_e}\right)$$

with $\beta_e > 0$ and finite $D_j \ge 1$, and candidate j violates that edge with $m_j \le -\delta_e < 0$. Then $\chi_j \le \exp(-\beta_e \delta_e / (\sqrt{D_j}\tau_e))$.

For a wrong-side set $A \subseteq N$, suppose every $j \in A$ violates some incident structural edge whose weight is at least $\beta_{\min} > 0$, whose signed margin is at most $-\delta_e$, and whose incident degree is at most $D_{\max}$. If at least one eligible reference candidate r has $\chi_r \ge \chi_{\min} > 0$, and the distance score ratio between every $j \in A$ and r is bounded by C, then the total wrong-side routing mass is at most

$$\frac{C|N|}{\chi_{\min}} \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Proof. If $j^\star_{\text{hard}} \notin N$, then the soft normalization assigns weight only to candidates in N and the omitted hard-route concept has no route to receive weight. This is the candidate-recall caveat.

For the nearest-reference statement,

$$\frac{\sum_{j\neq j^\star}\exp(-d_j/T)}{\exp(-d_{j^\star}/T)} = \sum_{j\neq j^\star}\exp(-(d_j-d_{j^\star})/T) \le (|N|-1)\exp(-\delta/T).$$

Since

$$1-\alpha_{j^\star} = \frac{\sum_{j\neq j^\star}\exp(-d_j/T)}{\exp(-d_{j^\star}/T)+\sum_{j\neq j^\star}\exp(-d_j/T)},$$

the same upper bound follows.

For an edge violation, set $a=\delta_e/\tau_e>0$. Since

$$\mathrm{sigmoid}(-a) = \frac{1}{1+\exp(a)} \le \exp(-a),$$

the corresponding log-compatibility contribution is at most $-\beta_e\delta_e/(\sqrt{D_j}\tau_e)$, and therefore $\chi_j \le \exp(-\beta_e\delta_e/(\sqrt{D_j}\tau_e))$. If A is the wrong-side set and r is an eligible candidate whose compatibility satisfies $\chi_r \ge \chi_{\min}$, the uniform wrong-side assumptions give, for every $j\in A$,

$$\chi_j \le \exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).$$

Combining this with the assumed distance-score ratio bound gives

$$\begin{aligned}\sum_{j\in A}\alpha_j &\le \sum_{j\in A}\frac{C}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right)\\ &\le \frac{C|N|}{\chi_{\min}}\exp\left(-\frac{\beta_{\min}\delta_e}{\sqrt{D_{\max}}\tau_e}\right).\end{aligned}$$

□

Theorem K.5 (Refine-or-repair objective descent, formal version of Theorem 3.7)**.** *Let*

$$J_{\mathcal{B}_t^{\mathrm{score}}}(\mathcal{S}) = \widehat{R}(\mathcal{S};\mathcal{B}_t^{\mathrm{score}}) + \lambda_{\mathrm{concept}}N_{\mathrm{concept}}(\mathcal{S}) + \lambda_{\mathrm{edge}}\,|E(\mathcal{S})|$$

with nonnegative penalties on a fixed scoring buffer. If a repair step is committed only after the guarded test

$$\widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\mathrm{score}}) \le \widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\mathrm{score}}),$$

and leaves anchors, edges, and concept count unchanged, then $J_{\mathcal{B}_t^{\mathrm{score}}}$ does not increase. If a refinement creates one concept and q edges and is accepted only when

$$\Delta_{\mathrm{refine}} > \max\{\widehat{\Delta}_{\mathrm{repair}},0\} + \tau + 2\epsilon_m,$$

where $\widehat{\Delta}_{\mathrm{repair}} \ge 0$ is the empirical repair gain and

$$\Delta_{\mathrm{refine}} = \widehat{R}(\mathcal{S}_t;\mathcal{B}_t^{\mathrm{score}}) - \widehat{R}(\mathcal{S}_{t+1};\mathcal{B}_t^{\mathrm{score}}) - \lambda_{\mathrm{concept}} - \lambda_{\mathrm{edge}}q,$$

then

$$J_{\mathcal{B}_t^{\mathrm{score}}}(\mathcal{S}_{t+1}) < J_{\mathcal{B}_t^{\mathrm{score}}}(\mathcal{S}_t) - \widehat{\Delta}_{\mathrm{repair}} - \tau - 2\epsilon_m.$$

In particular, $J_{\mathcal{B}_t^{\mathrm{score}}}$ decreases by more than $\tau+2\epsilon_m$.

Proof. For repair, N_{concept} and $|E|$ are unchanged. The empirical-risk term on $\mathcal{B}_t^{\text{score}}$ is no larger by the guard, so $J_{\mathcal{B}_t^{\text{score}}}$ does not increase. The identity candidate is always feasible, so the guarded rule can decline harmful centroid or memory movement instead of committing it.

For refinement,

$$\begin{aligned} J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_t) - J_{\mathcal{B}_t^{\text{score}}}(\mathcal{S}_{t+1}) &= \widehat{R}(\mathcal{S}_t; \mathcal{B}_t^{\text{score}}) - \widehat{R}(\mathcal{S}_{t+1}; \mathcal{B}_t^{\text{score}}) - \lambda_{\text{concept}} - \lambda_{\text{edge}} q \\ &= \Delta_{\text{refine}} \\ &> \widehat{\Delta}_{\text{repair}} + \tau + 2\epsilon_m. \end{aligned}$$

Rearranging gives the claim. □

The theorem is local to the fixed scoring buffer used in its statement. If the scoring buffer changes over time, these inequalities alone do not define a common global descent potential.

Theorem K.6 (Statistically safe finite-class refinement, formal version of Theorem 3.8)**.** *Assume losses lie in* $[0, 1]$*. Let* $\mathcal{H}$ *be a finite class of candidate refinements fixed before scoring, or generated from a proposal buffer independent of a scoring buffer* $\mathcal{B}^{\text{score}}$ *containing* m *independent samples. For candidate* g*, let* $\widehat{\Delta}(g)$ *be the empirical gain on* $\mathcal{B}^{\text{score}}$ *and let* $\Delta(g)$ *be the corresponding population gain. Then, with probability at least* $1 - \delta$*, every* $g \in \mathcal{H}$ *satisfies*

$$\left|\widehat{\Delta}(g) - \Delta(g)\right| \le 2\epsilon_m, \qquad \epsilon_m = \sqrt{\frac{\log(2(|\mathcal{H}| + 1)/\delta)}{2m}}.$$

Consequently, if

$$\widehat{\Delta}(g) > \lambda \operatorname{cost}(g) + \tau + 2\epsilon_m,$$

then on the same event

$$\Delta(g) > \lambda \operatorname{cost}(g) + \tau.$$

Proof. Apply Hoeffding's inequality to the empirical risk of the old state and to the empirical risk of each candidate-refined state. For any fixed state in this finite collection,

$$\mathbb{P}\left(\left|\widehat{R} - R\right| > \epsilon\right) \le 2\exp(-2m\epsilon^2).$$

Because $\mathcal{H}$ is fixed before scoring or generated from data independent of $\mathcal{B}^{\text{score}}$, Hoeffding's inequality applies conditionally on $\mathcal{H}$. Taking a union bound over the old state and the $|\mathcal{H}|$ candidate states gives the displayed ϵ_m. On this event, each gain is a difference of two risks, so empirical and population gain differ by at most $2\epsilon_m$. The implication follows by subtracting this deviation from the empirical gain lower bound. □

Corollary K.7 (Bounded language-model refinement score, formal version of Corollary 3.9)**.** *For language modeling, let*

$$\ell_{\text{clip}}(x_{\le t}, x_{t+1}) = \frac{1}{L_{\max}} \min\{-\log p(x_{t+1} \mid x_{\le t}), L_{\max}\}$$

for a fixed $L_{\max} > 0$*. If the scoring buffer, empirical gain, and population gain in Theorem K.6 are computed using* ℓ_{clip}*, then the theorem applies to language-model refinement decisions made with this clipped score. In particular, an accepted refinement has positive clipped population gain after paying complexity cost and margin* τ *with probability at least* $1 - \delta$*.*

Proof. The clipped score satisfies $0 \le \ell_{\text{clip}} \le 1$. Substituting this bounded loss into Theorem K.6 gives the claim directly. The statement is only about the clipped score; raw negative log likelihood is unbounded and would require a separate concentration assumption. □

Theorem K.8 (Anchor-stable DGM information structure, formal version of Theorem 3.10)**.** *Let*

$$\mathcal{F}_t^{\text{DGM}} = \sigma(g_e : e \in E_t) \vee \sigma\Big(\{h : \|h - a_i\|_2^2 \le \|h - a_j\|_2^2\} : 1 \le i, j \le N_t\Big).$$

Suppose every edge generator $g_e(h) = \mathbf{1}\{u_e^\top h > b_e\}$ is computed from frozen endpoint anchors a_i, a_j. Any repair step that changes only centroids c_j, local memories M_j, and counts n_j, while keeping anchors and edges fixed, leaves $\mathcal{F}_t^{\text{DGM}}$ unchanged. If a refinement adds a new anchor and new edge generators, then $\mathcal{F}_{t+1}^{\text{DGM}}$ is $\mathcal{F}_t^{\text{DGM}}$ joined with the new anchor-order events and the new edge generators.

Proof. For an edge $e = (i, j)$, both

$$u_e = \frac{a_i - a_j}{\|a_i - a_j\|_2}, \qquad b_e = \frac{1}{2} u_e^\top (a_i + a_j)$$

are deterministic functions of the frozen anchors. A repair step does not change these anchors, so it does not change u_e, b_e, or g_e. It also does not change any anchor-distance comparison event because all anchors are fixed. Hence both sigma-algebras in the join defining $\mathcal{F}_t^{\text{DGM}}$ are unchanged.

If a refinement adds anchor a_{new} and edge generators E_{new}, the new edge sigma-algebra is $\sigma(g_e : e \in E_t) \vee \sigma(g_e : e \in E_{\text{new}})$, and the new anchor-order sigma-algebra is the old anchor-order sigma-algebra joined with the pairwise comparisons involving a_{new}. This is exactly the stated join. □

Corollary K.9 (Finite number of accepted refinements, formal version of Corollary 3.11)**.** *Let Φ be a common potential with $\Phi(\mathcal{S}) \ge 0$ for all reachable states. If each accepted refinement decreases Φ by at least $\tau > 0$, then the number of accepted refinements is at most $\Phi(\mathcal{S}_0)/\tau$.*

Proof. After m accepted refinements,

$$\Phi(\mathcal{S}_m) \le \Phi(\mathcal{S}_0) - m\tau.$$

Since $\Phi(\mathcal{S}_m) \ge 0$, we have $m \le \Phi(\mathcal{S}_0)/\tau$. □

Proposition K.10 (Trees as generalized hard-route DGMs, formal version of Proposition D.1)**.** *Let $\mathcal{T}$ be a binary decision tree whose internal nodes compute binary distinctions $g_\nu(h) \in \{0, 1\}$ and whose leaves have predictors M_ℓ. Construct a generalized hard-route DGM with one concept c_ℓ and memory M_ℓ for each leaf ℓ, where the generalized model permits arbitrary finite binary eligibility constraints in addition to pairwise distinction edges. For each leaf, add hard eligibility constraints equal to the signed distinctions along the root-to-leaf path. If the generalized hard route selects the unique eligible leaf concept, then its prediction equals the tree prediction for every h.*

Proof. For any input h, the tree follows exactly one root-to-leaf path. The leaf reached by the tree is the unique leaf whose path constraints are all satisfied by h. By construction, the corresponding generalized hard-route concept has exactly those eligibility constraints, and all other leaf concepts violate at least one path constraint at the first node where their path differs from the tree path. Therefore the generalized hard route selects the same leaf and applies the same memory M_ℓ. □

Theorem K.11 (DGM data-structure bound, formal version of Theorem D.2)**.** *Assume DGM has N_t concepts in $\mathbb{R}^d$, exact centroid and anchor retrieval, candidate set $N(h)$, selected incident edge-gate count $Q(h) = \sum_{j \in N(h)} |I_t(j)|$, and local-memory evaluation cost C_{local}. Then one prediction costs $O(N_t d + Q(h) + |N(h)|d + C_{\text{local}})$. One observation that repairs a selected concept costs this plus the guarded local repair scoring cost. One observation that creates a new anchor-centroid pair and q distinction edges costs an additional $O(d + qd)$. If each concept stores an anchor, a centroid, and an embedding mean in $\mathbb{R}^{d_e}$, and each edge stores a linear distinction in $\mathbb{R}^d$, then the runtime state uses $O(N_t(2d + d_e) + |E_t|d)$ memory.*

Proof. A distance computation between $h \in \mathbb{R}^d$ and one concept center costs $O(d)$. Exact centroid and anchor retrieval over N_t concepts therefore costs

$$O(N_t d).$$

Graph-neighbor expansion has cost proportional to the incident lists touched and is absorbed into selected edge-gate bookkeeping. After the candidate set $N(h)$ is selected, evaluating the incident edge gates costs $O(Q(h))$. Computing sparse routing weights, distances within the selected set, and evaluating selected local memories costs

$$O(|N(h)|d + C_{\text{local}}).$$

Thus prediction costs

$$O(N_t d + Q(h) + |N(h)|d + C_{\text{local}}).$$

Observation begins with the same retrieval computation. If it repairs a selected concept, it adds the cost of evaluating and scoring the proposed local repair candidates on the scoring buffer. If it creates a new anchor-centroid pair, storing the pair costs $O(d)$. For each new edge, computing and storing the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

The total additional cost for refinement is therefore $O(d + qd)$.

For memory, each concept stores $a_j, c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, giving

$$O(N_t(2d + d_e)).$$

Each edge stores one distinction vector and threshold, giving

$$O(|E_t|d).$$

Adding the two terms proves the claimed bound. □

Theorem K.12 (DGM layer cost, formal version of Theorem D.3)**.** *Consider a DGM adapter extension with hidden dimension d, query dimension d_q, local rank r, candidate count $|N|$, selected incident edge-gate count Q_N, and concept-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|dr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|(d_q + dr)).$$

For a block of length L, backpropagation through the slow parameters does not require storing the full DGM state history $\mathcal{S}_1, \ldots, \mathcal{S}_L$.

Proof. For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition. Evaluating selected incident edge gates costs $O(Q_N)$.

For a selected concept j, the rank-r adapter read is

$$W_j K h.$$

Multiplying one selected $W_j \in \mathbb{R}^{d\times r}$ by a rank-r key costs $O(dr)$. Therefore $|N|$ selected adapters cost

$$O(|N|dr).$$

The routing weights add lower-order top-k normalization cost, absorbed into the same bound. This proves the read overhead.

The write path first computes the same query, retrieval, and edge gates. It then updates selected concept statistics and selected local memories. Concept bookkeeping costs $O(d_q)$ per selected concept. A rank-one local repair of a rank-r or low-rank sufficient-statistic memory costs $O(dr)$ per selected memory. Hence the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + Q_N + |N|(d_q + dr)).$$

Finally, the update rule is applied as

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

Thus the computational graph for the loss contains the selected read computation, but not the chain of state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$. Therefore full DGM state history is not required for backpropagation through the slow parameters. □

Theorem K.13 (One-hot memory is partition-measurable, formal version of Theorem H.1)**.** *Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}}(\omega) = e_i$ for $\omega \in B_i$. For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is equal to f_W for some $W \in \mathbb{R}^{m\times n}$.*

Proof. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

so f_W is constant on each atom and is $\mathcal{P}$-measurable.

Conversely, if f is $\mathcal{P}$-measurable, it is constant on each B_i. Let w_i be this constant value on B_i, and define

$$W = [w_1, \ldots, w_n].$$

Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

□

Theorem K.14 (CPM conflict equals missing distinction, formal version of Theorem H.2)**.** *Let* $\mathcal{P} = \{B_1, \ldots, B_n\}$ *and let* $\omega, \omega' \in B_i$*. Suppose* $v(\omega) \neq v(\omega')$*. Let* $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$*. If a memory* W *has already committed* $We_i = v(\omega)$*, then the evidence* $We_i = v(\omega')$ *has innovation* $z = 0$ *relative to the old key span and residual* $e \neq 0$*.*

Proof. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

The old key span contains e_i. Therefore the innovation of the new key $k = e_i$ relative to the old key matrix $K = e_i$ is

$$z = (I - KK^{\dagger})k = (I - e_i e_i^{\top})e_i = 0.$$

The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □